Politechnika Warszawska

Warsaw University of Technology

https://repo.pw.edu.pl

Tytuł / Title	Porównanie metod wstępnego przetwarzania i klasyfikacji danych biomedycznych / Comparison of preprocessing and classification methods of biomedical data
Autor / Author	Tąkiel Piotr
Promotor / Supervisor	Nowak Robert Marek
Adres publikacji w Repozytorium URL / Publication address in Repository	https://repo.pw.edu.pl/info/master/WUT6053bccc0f3741969d67c35c726c91ec/
Data opublikowania w Repozytorium / Deposited in Repository on	2023-11-14

Rok akademicki 2014/2015

Politechnika Warszawska
Wydział Elektroniki i Technik Informacyjnych
Instytut Informatyki

PRACA DYPLOMOWA MAGISTERSKA

Piotr Tąkiel

Porównanie metod wstępnego przetwarzania i klasyfikacji danych biomedycznych

Opiekun pracy
dr inż. Robert Marek Nowak

Ocena: .....

.....
Podpis Przewodniczącego
Komisji Egzaminu Dyplomowego

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Kierunek: Informatyka

Specjalność: Inżynieria Systemów Informatycznych

Data urodzenia: 12.01.1990

Data rozpoczęcia studiów: 01.10.2009

Życiorys

Urodziłem się 12 stycznia 1990 r. w Warszawie. Uczęszczałem do Szkoły Podstawowej Nr 218 im. Michała Kajki oraz Nr 173 im. Górników Polskich w Warszawie. Ukończyłem Gimnazjum Nr 119 im. Józefa Piłsudskiego oraz Liceum Nr 14 im. Stanisława Staszica w Warszawie. W 2009 r. zostałem studentem Wydziału Elektroniki i Technik Informacyjnych Politechniki Warszawskiej. W 2012 r. semestr studiów spędziłem na uczelni zagranicznej Kungliga Tekniska Hogskolan w Szwecji.

.....
Podpis studenta

EGZAMIN DYPLOMOWY

Złożył egzamin dyplomowy w dniu ..... 20__ r

z wynikiem .....

Ogólny wynik studiów: .....

Dodatkowe wnioski i uwagi Komisji: .....

.....

.....

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

STRESZCZENIE

Porównanie metod wstępnego przetwarzania i klasyfikacji danych biomedycznych. Celem pracy jest stworzenie systemu komputerowego służącego do porównywania rodzin algorytmów przetwarzania wstępnego i klasyfikacji danych. Aplikacja była testowana na danych dotyczących zachorowalności na raka piersi u kobiet. Praca obejmuje wprowadzenie teoretyczne do zagadnień związanych z uczeniem maszynowym, wymagania postawione systemowi, projekt rozwiązania oraz elementy implementacji. Praca kończy się wnioskami wyciągniętymi z analizy otrzymanych wyników.

Słowa kluczowe: klasyfikacja, przetwarzania wstępne, uczenie maszynowe, obliczenia rozproszone

---

Comparison of preprocessing and classification methods of biomedical data

The thesis purpose is to create a computer system comparing families of preprocessing and classification algorithms. The system was tested on women breast cancer data. The thesis includes introduction to machine learning, system requirements, system project and detailed elements of system implementation. The thesis ends with conclusions coming from the analysis of the computed results.

Keywords: classification, preprocessing, machine learning, distributed computing

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Spis treści

1. Wstęp . . . 5
1.1 O pracy . . . 5
1.2 Cel i zakres pracy . . . 6
1.2.1 Cel pracy . . . 6
1.2.2 Zakres pracy . . . 6
2. Wprowadzenie teoretyczne . . . 8
2.1 Klasyfikacja . . . 8
2.1.1 Ocena klasyfikacji . . . 8
2.1.2 Nadmierne dopasowanie . . . 9
2.1.3 Przegląd algorytmów klasyfikacji . . . 10
2.2 Przetwarzanie wstępne . . . 12
2.2.1 Transformacje danych . . . 12
2.2.2 Brakujące wartości atrybutów . . . 13
2.2.3 Redukcja wymiarów . . . 14
2.2.4 Modele przetwarzania wstępnego . . . 15
2.3 Testy statystyczne . . . 15
2.4 Podsumowanie . . . 15
3. Założenia i wymagania systemu . . . 17
3.1 Wymagania funkcjonalne . . . 17
3.1.1 Wprowadzanie danych wejściowych . . . 17
3.1.2 Wykonywanie obliczeń . . . 22
3.1.3 Prezentacja wyników . . . 24
3.2 Wymagania niefunkcjonalne . . . 24
3.2.1 Rozmiar przechowywanych danych . . . 24
3.2.2 Dane wejściowe . . . 24
3.2.3 Algorytmy uczenia maszynowego . . . 25
3.3 Wymagania systemowe . . . 25
3.4 Wymagania ograniczeń . . . 25
3.5 Podsumowanie . . . 25
4. Projekt systemu komputerowego . . . 27
4.1 Architektura systemu . . . 27
4.1.1 Przegląd rozwiązań . . . 27
4.1.2 Opis architektury . . . 28
4.2 Biblioteka obliczeniowa mltool . . . 28
4.2.1 Struktura biblioteki . . . 28
4.2.2 Odczyt danych . . . 29
4.2.3 Modele przetwarzania i klasyfikacji . . . 30
4.2.4 Deskryptory operacji . . . 32
4.2.5 Generowanie algorytmów wewnątrz rodzin . . . 35
4.3 Struktura przechowywanych danych . . . 40
4.4 Program nadzorujący obliczenia . . . 40
4.4.1 Interfejs użytkownika . . . 40
4.4.2 Zarządzenie obliczeniami równoległymi . . . 41
4.4.3 Warstwa dostępu do danych . . . 43

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

4.5 Program liczący . . . 44
4.6 Prezentowanie wyników obliczeń . . . 44
4.6.1 Program browse . . . 44
4.6.2 Program compare . . . 45
4.7 Testy . . . 46
5. Elementy implementacji systemu . . . 48
5.1 Użyte technologie . . . 48
5.1.1 Język programowania Python . . . 48
5.1.2 System kolejkowy RabbitMQ . . . 48
5.1.3 Biblioteka Celery . . . 49
5.1.4 Baza danych PostgreSQL . . . 49
5.1.5 Mapowanie obiektowo-relacyjne SQLAlchemy . . . 49
5.1.6 Biblioteka scikit-learn . . . 50
5.2 Program liczący . . . 50
5.2.1 Buforowanie wyników . . . 50
5.2.2 Wykonywanie obliczeń . . . 52
6. Badania . . . 54
6.1 Przebieg badań . . . 54
6.1.1 Faza pierwsza . . . 54
6.1.2 Faza druga . . . 57
6.2 Wnioski z badań . . . 58
7. Podsumowanie . . . 60
Bibliografia . . . 61

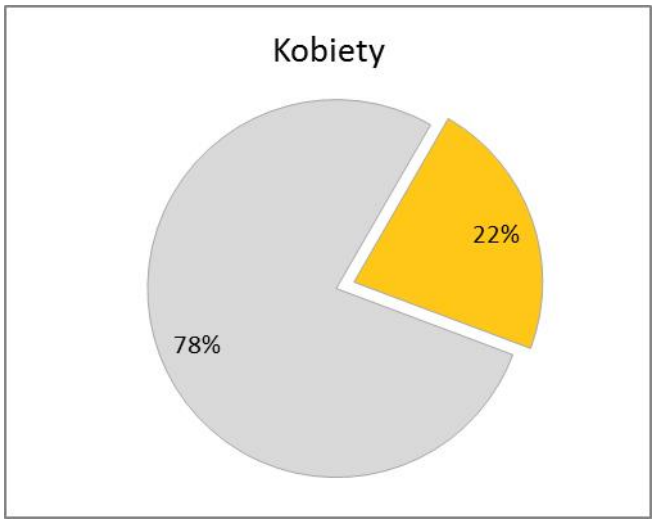
Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

1. Wstęp

1.1 O pracy

Tematem mojej pracy magisterskiej jest porównanie metod wstępnego przetwarzania i klasyfikacji danych biomedycznych. Temat ten wydaje mi się ciekawy z wielu powodów.

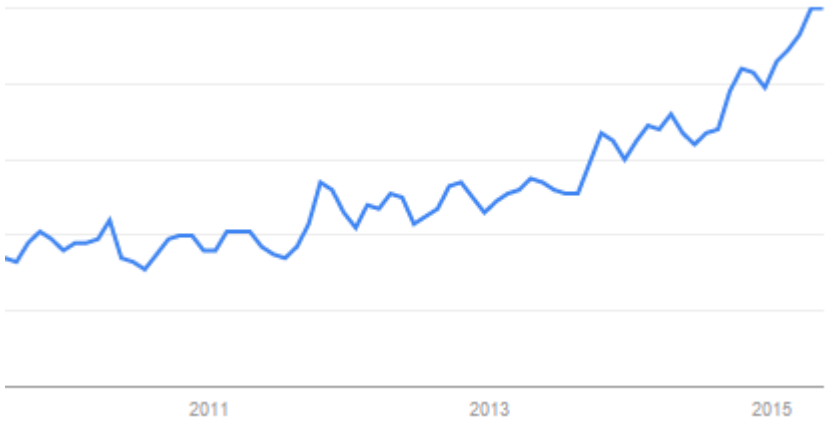
Podstawowym powodem, dla którego postanowiłem zająć się tym tematem, jest aktualność problemu raka piersi u kobiet. Zapadalność na raka piersi wzrasta z roku na rok w krajach wysoko rozwiniętych. W krajach tych, pomimo postępów w diagnozowaniu i leczeniu, obserwuje się wzrost corocznej liczby zgonów wywołanej tą chorobą. W Polsce kobiety chorują na raka piersi nieco rzadziej niż w reszcie świata. U co dwunastej kobiety na świecie diagnozuje się raka piersi, podczas gdy w Polsce jest to co czternasta kobieta [1]. Wyleczalność wygląda w naszym kraju znacznie gorzej. Co druga polka chora na raka piersi umiera z powodu tej choroby, podczas gdy np. w Holandii jedynie trzy na dziesięć kobiet. Wg badań z 2010 r. nowotwory złośliwe piersi stanowią u Polek 22% wszystkich zachorowań nowotworowych [2].



Rysunek 1.1: Częstość zachorowań na nowotwory piersi w Polsce w 2010 roku

Drugiem powodem, dla którego postanowiłem zająć się tą pracą, jest popularność metod uczenia maszynowego. Uczenie maszynowe znajduje bardzo wiele praktycznych zastosowań, w skład których wchodzą: rozpoznawanie obrazów, wyszukiwanie, detekcja spamu, wykrywanie fałszerstw finansowych oraz rozpoznawanie chorób na podstawie symptomów.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00



Rysunek 1.2: Zainteresowanie hasłem *machine learning* wg wyszukiwarki Google Trends

1.2 Cel i zakres pracy

1.2.1 Cel pracy

Celem mojej pracy magisterskiej jest zaprojektowanie oraz implementacja systemu komputerowego umożliwiającego porównywanie jakości klasyfikacji rozmaitych algorytmów przetwarzania wstępnego i klasyfikacji danych medycznych. Algorytmy będą badane na danych medycznych. Efektem działania systemu powinien być ranking zbiorów algorytmów, które wprowadził użytkownik. System powinien oferować prosty interfejs użytkownika pozwalający mu na elastyczny wybór rodzin algorytmów do oceny. Część wykonująca obliczenia powinna być możliwie odseparowana od interfejsu, umożliwiając niezależną pracę nad tymi dwoma modułami.

Projekt systemu jest w pewnym stopniu oparty na danych, jakimi dysponowałem przez cały czas pisania pracy. Dostępne dane są rzeczywiste i anonimowe, opisują one zachorowalność na raka piersi u ok. 1000 pacjentek. Zbiór danych posiada sześć kolumn (atrybutów) zawierających informacje o wynikach badań pacjentek oraz kolumnę chorobową (kategorię) określającą, czy pacjentka była chora na nowotwór, czy nie.

1.2.2 Zakres pracy

Zakres mojej pracy można podzielić na część teoretyczną i praktyczną. Pierwsza z nich obejmuje przede wszystkim wprowadzenie do problemu klasyfikacji danych w ujęciu uczenia maszynowego, sformułowanie wymagań dotyczących rozwiązania, projekt systemu oraz wnioski z pracy. Druga część koncentruje się na realizacji zaplanowanej architektury, dokładnej specyfikacji niektórych fragmentów implementacji oraz na opisach testów systemu.

Rozdział następny stanowi wprowadzenie do problemu klasyfikacji oraz przetwarzania wstępnego danych w kontekście uczenia maszynowego. Po lekturze tego rozdziału czytelnik powinien mieć podstawową wiedzę na temat tego, jak działają algorytmy sztucznej inteligencji użyte w mojej pracy.

Rozdział trzeci zatytułowany *Założenia i wymagania systemu* dotyczy wymagań stawianych projektowanemu systemowi obliczeń. Opracowanie wymagań jest bardzo ważnym elementem pracy, ponieważ, poprzez analizę tego co wykonalne, pozwala zrozumieć, jak powinien wyglądać projektowany system. Ze względu na prototypowy typ systemu, praca skupia się na wymaganiach funkcjonalnych i ograniczeniach narzędziowych oraz implementacyjnych.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Rozdział pt. *Projekt systemu komputerowego* zawiera ideowe opisy modułów programowych użytych w systemie. W rozdziale tym można znaleźć diagramy klas, opisy funkcji, metod i architektury systemu. Projekt rozwiązania jest przygotowany niezależnie od późniejszej implementacji, dając autorowi (oraz innym osobom) swobodę w realizacji systemu.

Rozdział kolejny zatytułowany *Elementy implementacji systemu* pokazuje zasadę działania wybranych, najciekawszych elementów systemu. W części tej znajdują się fragmenty kodu źródłowego wraz z objaśnieniami.

Wyniki badań są zgromadzone w rozdziale szóstym. Wnioski płynące z analizy wyników badań oraz z całości pracy nad systemem umieszczone są w ostatnim rozdziale pracy.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

2. Wprowadzenie teoretyczne

Celem tego rozdziału jest dostarczenie opisu podstawowych zagadnień związanych z uczeniem maszynowym. W szczególności rozdział ten poświęcony jest dwóm dziedzinom wiedzy: przetwarzaniu wstępnemu oraz (późniejszej) klasyfikacji danych. Końcowa część rozdziału poświęcona jest testom statystycznym, a szczególnie testowi Manna-Whitneya-Wilcoxona.

2.1 Klasyfikacja

Klasyfikacja statystyczna to rodzaj algorytmu, który przydziela obserwacje statystyczne do klas, bazując na atrybutach tych obserwacji [3]. Algorytm klasyfikacji tworzy model klasyfikacji na podstawie danych trenujących zawierających znane klasy (kategorie). Gotowy model można następnie zastosować do predykcji kategorii nowych, nieznanych wcześniej, obserwacji. Klasyfikacja jest zatem przykładem uczenia się z nadzorem. Nadzór polega na obecności w zbiorze trenującym informacji o kategoriach danych, które mamy. Algorytm tworzenia modelu stara się nauczyć model docelowych kategorii, jednocześnie dbając o to, aby wiedza na temat problemu była jak najbardziej ogólna i niezależna od próby, którą dysponujemy.

x1	x2	c(x)	*h(x)*
1	0,56	1	*1*
1	2,5	1	*1*
2	2,13	1	*0*
1	1,14	1	*0*
3	0,75	1	*0*
2	1,15	0	*0*
1	2,53	0	*0*
1	1,98	0	*0*
3	4,3	0	*1*

Rysunek 2.1: Przykład klasyfikacji danych o atrybutach x_1, x_2 oraz kategorii $c(x)$. Kolumna $h(x)$ oznacza hipotezę klasyfikatora (predykcję). Dwa pierwsze przykłady zostały zaklasyfikowane poprawnie, trzy następnie niepoprawnie, następne trzy poprawnie, a ostatni nieprawidłowo.

2.1.1 Ocena klasyfikacji

W celu oceny gotowego modelu klasyfikacji, możemy posłużyć się miarami zdefiniowanymi na podstawie zawartości tzw. macierzy pomyłek. Dla klasyfikacji dwuwartościowej macierz pomyłek składa się z dwóch wierszy i dwóch kolumn. Kolumny odpowiadają faktycznym kategoriom danych, wiersze zaś predykcjom klasyfikatora. Na przecięciu kolumn i wierszy znajdują się cztery wartości liczbowe charakteryzujące poprawność wyników modelu. Wartości *True Positive* i *True Negative* określają ile poprawnych wyników pozytywnych i negatywnych zostało zwróconych przez klasyfikator. Z kolei wartości *False Positive* i *False Negative* pokazują, ile pozytywnych i negatywnych przypadków model zaklasyfikował niewłaściwie.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

		Stan faktyczny	
		0	1
Odpowiedź klasyfikatora	0	TN	FN
	1	FP	TP

Rysunek 2.2: Przykładowa macierz pomyłek.

Ocenę jakości modelu klasyfikacji łatwo wyrazić wartościami liczbowymi wyliczanymi w oparciu na poszczególnych komórkach macierzy pomyłek. Celnością c nazywamy stosunek liczby poprawnych wyników klasyfikacji do wszystkich rozpatrywanych przypadków:

$$c = \frac{t_p + t_n}{t_p + t_n + f_p + f_n}$$

gdzie t_p, t_n, f_p i f_n oznaczają odpowiednio wartości *True Positive*, *True Negative*, *False Positive* i *False Negative*. Miara ta pokazuje, jak często klasyfikator daje poprawny wynik. W medycynie częściej jednak stosuje się inne miary oceny, w szczególności *precyzję* i *wrażliwość*.

Precyzja odpowiada na pytanie, jaka część zdiagnozowanych przypadków była prawdziwa. Precyzję p wyrażamy jako stosunek liczby poprawnie zdiagnozowanych przypadków do liczby wszystkich pozytywnych wyników predykcji modelu:

$$p = \frac{t_p}{t_p + f_p}$$

gdzie t_p i f_p oznaczają odpowiednio wartości *True Positive* i *False Positive*.

Druga miara - wrażliwość - pokazuje nam, jaka część chorych została wykryta przez klasyfikator. Wrażliwość w wyraża się jako iloraz liczby słusznie zdiagnozowanych przypadków oraz liczby wszystkich zachorowań:

$$w = \frac{t_p}{t_p + f_n}$$

gdzie t_p i f_n oznaczają odpowiednio wartości *True Positive* i *False Negative*.

W praktyce trudno jednak o model, który ma jednocześnie wysokie wartości precyzji i wrażliwości. Z tego powodu często stosuje się miarę *F1*, będącą średnią harmoniczną dwóch powyższych miar:

$$f_1 = \frac{2t_p}{2t_p + f_p + f_n}$$

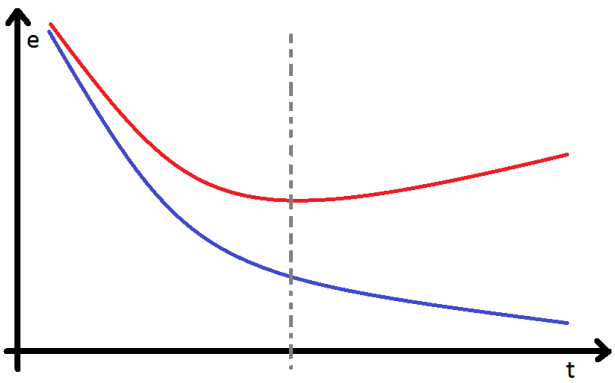
gdzie f_1, t_p, f_p i f_n to odpowiednio wartość miary *F1*, a następnie wartości *True Positive*, *False Positive* i *False Negative*.

2.1.2 Nadmierne dopasowanie

Oceniając jakość klasyfikacji nie należy zapominać o problemie nadmiernego dopasowania modelu. Nadmierne dopasowanie to zjawisko zachodzące, gdy model klasyfikacji daje gorsze wyniki dla nowych danych niż dla znanych danych trenujących. Aby ocena modelu nie była obarczona błędem związanym z tym zjawiskiem, do ewaluacji klasyfikatora należy używać osobnego zbioru testowego rozłącznego ze zbiorem trenującym.

Dzieląc dostępne dane na zbiór trenujący i testowy należy pamiętać o konsekwencjach tego podziału. Im mniejszy jest zbiór uczący (a testowy większy) tym mniej informacji stosujemy do nauki modelu i końcowa ocena może być zaniżona. Z drugiej strony, im mniejszy

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00



Rysunek 2.3: Wykresy zależności błędu klasyfikatora od czasu związanego z nauką. Linia niebieska prezentuje błąd na zbiorze trenującym, który nieprzerwanie maleje z czasem. Wykres czerwony przedstawia błąd rzeczywisty na całej populacji danych. Nadmierne dopasowanie zaczyna się od momentu zaznaczonego szarą linią - błąd rzeczywisty zaczyna rosnąć.

jest zbiór testowy, tym mniej jest reprezentatywny dla całej populacji i tym większa będzie wariancja oceny na tym zbiorze.

Z powyższym problemem można radzić sobie wykonując wiele testów. W literaturze wyróżnia się zwykle dwie metody z tym związane: *sprawdzian krzyżowy* (inaczej: walidacja krzyżowa) oraz *metoda bootstrap*.

Sprawdzian krzyżowy polega na podziale danych na n równolicznych części, utworzeniu danych uczących z $n-1$ z nich i przeprowadzeniu testów na pozostałej części. Uczenie i ocenę modelu powtarza się n-krotnie, za każdym razem dla innych części. Otrzymane n ocen można użyć np. do wyliczenia oceny końcowej, będącej średnią.

Metoda bootstrap tworzy zbiór uczący poprzez losowanie ze zwracaniem ze zbioru danych wejściowych. Przykłady niewylosowane stają się zbiorem testowym.

2.1.3 Przegląd algorytmów klasyfikacji

Do najpopularniejszych klasyfikatorów stosowanych obecnie należą naiwny klasyfikator bayesowski, drzewo decyzyjne, las losowy oraz maszyna wektorów nośnych. Dalsza część rozdziału skupi się na omówieniu tych algorytmów.

Naiwny klasyfikator bayesowski

Naiwny klasyfikator bayesowski jest klasyfikatorem probabilistycznym. Oznacza to, że dla danego przykładu wynikiem zwracanym przez model są prawdopodobieństwa poszczególnych kategorii. Klasyfikator ten znany jest od ponad 60 lat – badania nad nim prowadzone były już w latach 50 XX w. Zasada działania tego klasyfikatora opiera się na twierdzeniu Bayesa, znanym z probabilistyki:

$$P(c_i|x) = \frac{P(c_i)P(x|c_i)}{P(x)}$$

gdzie $P(c_i|x)$ oznacza prawdopodobieństwo, że przykład x należy do kategorii c_i, $P(c_i)$ oznacza prawdopodobieństwo wystąpienia kategorii c_i w ogóle, $P(x|c_i)$ to prawdopodobieństwo wystąpienia przykładu x pod warunkiem zaistnienia kategorii c_i, a $P(x)$ to prawdopodobieństwo wystąpienia przykładu x.

Naiwność tej metody klasyfikacji polega na założeniu, że wewnątrz kategorii, atrybuty są względem siebie niezależne. W wielu przypadkach założenie to nie jest spełnione, co odbija

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

się na jakości oceny klasyfikatora. Mimo tej oczywistej wady, naiwny klasyfikator bayesowski znajduje praktyczne zastosowanie, np. przy diagnozowaniu niektórych chorób.

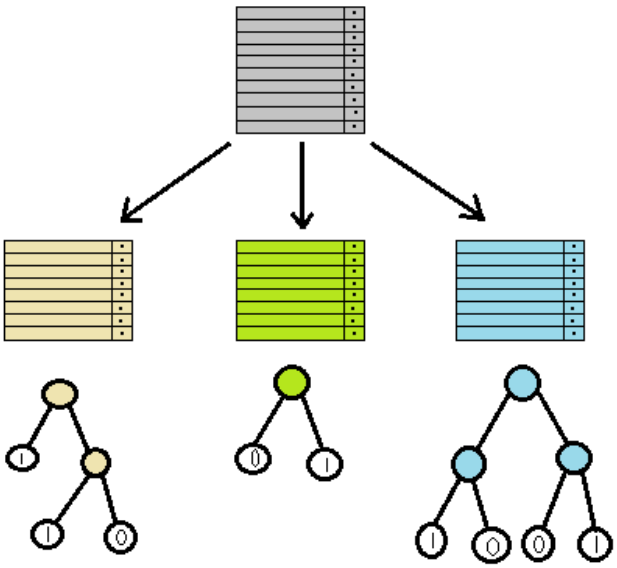
Drzewo decyzyjne

Kolejny popularnym klasyfikatorem jest drzewo decyzyjne. W trakcie uczenia modelu budowane jest drzewo, w którego węzłach znajdują się testy na wartości atrybutów, a w liściach kategorie lub prawdopodobieństwa kategorii. W trakcie predykcji przykład wędruje od korzenia drzewa do liścia, przechodząc konkretnymi gałęziami w zależności od wyników testów w węzłach. Popularność drzewa decyzyjnego bierze się m.in. z tego, że bardzo często daje ono zadowalające wyniki klasyfikacji.

Częstym problemem drzew decyzyjnych jest duży rozrost wgłąb i, towarzyszące temu, nadmierne dopasowanie do danych trenujących. Z problemem tym można walczyć, przycinając drzewo, tj. zastępując poddrzewa liśćmi. Postępowanie takie wynika z przekonania, że modele mniej skomplikowane lepiej generalizują wiedzę. Wśród algorytmów przycinania wyróżnia się m.in.: *Reduced Error Pruning*, *Minimum Error Pruning* oraz *Cost Complexity Pruning*.

Las losowy

Rozwinięciem pomysłu związanego z drzewami decyzyjnymi jest klasyfikator znany jako las losowy (ang. *random forest*). Klasyfikator ten to zbiór drzew decyzyjnych, z których każde nauczone jest na oddzielnym zbiorze trenującym uzyskanym *metodą boostrap*. Podczas predykcji przykład przechodzi przez każde z drzew, generując przy tym serię wyników. Rezultaty tych pomniejszych klasyfikacji grupowane są w zbiorowy ranking, na podstawie którego klasyfikator losowy las podejmuje ostateczną decyzję co do kategorii przykładu.



Rysunek 2.4: Poglądowa ilustracja przedstawiająca las losowy składający się z trzech drzew decyzyjnych.

Maszyna wektorów nośnych

Ostatnim klasyfikatorem omówionym w tej części pracy jest Maszyna Wektorów Nośnych. Jest to klasyfikator binarny. Nauka modelu klasyfikatora ma na celu wyznaczenie hiperpłaszczyzny rozdzielającej z maksymalnym marginesem przykłady należące do dwóch klas.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

2.2 Przetwarzanie wstępne

Celem przetwarzania wstępnego, w kontekście uczenia maszynowego i klasyfikacji, jest zamiana oryginalnej postaci danych wejściowych na taką, która umożliwia uzyskiwanie lepszych wyników późniejszej klasyfikacji [4]. W mojej pracy dyplomowej korzystam z pewnych metod wstępnego przetwarzania danych i badam ich wpływ na końcową ocenę klasyfikacji. W części tej omawiam użyte przeze mnie metody, jak również kilka innych.

2.2.1 Transformacje danych

Standaryzacja

Wiele algorytmów klasyfikacji oczekuje wartości atrybutów o rozkładzie przypominającym standardowy rozkład normalny. Dane, którymi dysponujemy, mogą być zgrupowane w innej postaci. Aby zapewnić dobre wyniki klasyfikacji, często stosuje się *standaryzację*. Metoda ta polega na przesunięciu danych o ich wartość średnią w kierunku zera i późniejszym podzieleniu przez wartość odchylenia standardowego:

$$x_i^{std} = \frac{x_i - \mu(X)}{\sigma(X)}$$

gdzie x_i^{std} oznacza nową wartość po standaryzacji, x_i wartość początkową, $\mu(X)$ wartość średnią atrybutu, a $\sigma(X)$ odchylenie standardowe atrybutu. Po wykonaniu powyższej operacji rozkład danych przypomina standardowy rozkład normalny.

Skalowanie

Innym rozwiązaniem jest *skalowanie* atrybutów. Polega ono na proporcjonalnym przeskalowaniu wartości atrybutu tak, aby znalazły się w ustalonym przedziale. Częstymi docelowymi przedziałami są [0, 1] i [-1, 1]. Przekształcenie opisuje następująca zależność:

$$x_i = r_s + (r_e - r_s)\frac{x_i - X_{min}}{X_{max} - X_{min}}$$

gdzie x_i oznacza nową wartość atrybutu po przeskalowaniu, r_s początek docelowego przedziału, r_e koniec docelowego przedziału, X_{min} minimalną wartość atrybutu, a X_{max} wartość maksymalną.

Normalizacja

Metoda *normalizacji* stanowi inne spojrzenie na modyfikowanie danych. Zamiast działać na atrybutach, normalizacja zajmuje się pojedynczymi przykładami z danych. Metoda ta polega na takim przeskalowaniu przykładu, aby jego długość (rozumiana w sensie interpretacji wektorowej) była równa 1. W praktyce oznacza to podzielenie wektora przykładu przez jego normę:

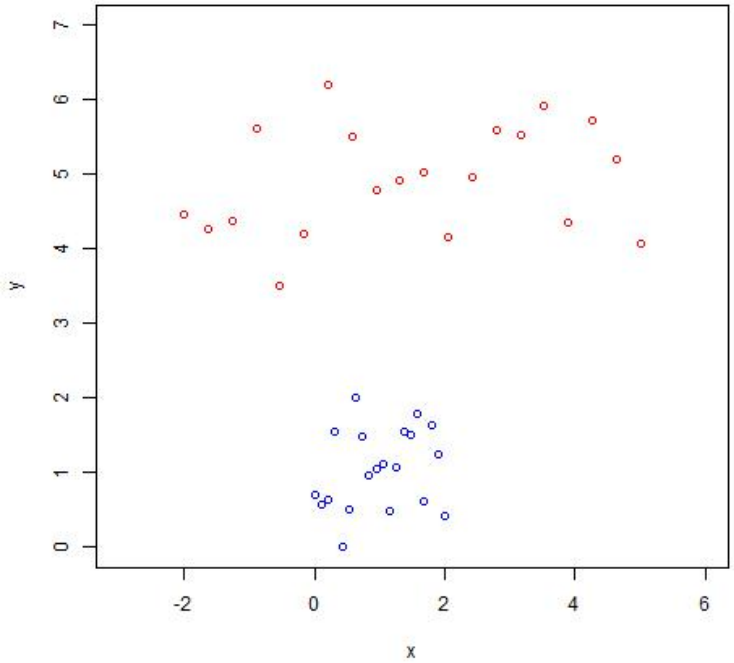
$$\vec{x_n} = \frac{\vec{x}}{\|\vec{x}\|}$$

gdzie $\vec{x_n}$ oznacza przykład znormalizowany, $\vec{x}$ przykład początkowy, a $\|\vec{x}\|$ normę wektora (przykładu).

Binaryzacja

Przedstawione do tej pory metody koncentrowały się na skalowaniu danych. W odróżnieniu od nich, *binaryzacja* polega na zamianie wartości atrybutów na zera i jedynki. Wybór

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00



Rysunek 2.5: Dane oryginale przed skalowaniem (czerwone punkty) oraz po przeskalowaniu do zakresu [0, 2] w osiach X i Y (niebieskie punkty).

pomiędzy dwoma liczbami opiera się na wartości ustalonego progu. Wszystkie wartości atrybutów większe od progu zamieniane są na jedynki, pozostałe zaś – na zera. Po zastosowaniu binaryzacji każdy przykład stanowi pewną kombinację zer lub jedynek. Pozwala to nam interpretować przykłady jako realizacje schematu Bernoulliego, gdzie jedynka oznacza sukces, a zero porażkę.

2.2.2 Brakujące wartości atrybutów

Cztery metody opisane w poprzednim punkcie zakładają, że wśród danych nie ma brakujących wartości. W realnych zastosowaniach często niestety jest inaczej – dane, którymi dysponujemy, mają pewien odsetek braków. Przykładowo, dane medyczne użyte w mojej pracy mają 8% brakujących wartości. Nawet, jeżeli przyjmiemy, że wyżej wymienionych metod nie będzie się stosować, to wciąż niektóre algorytmy klasyfikacji wymagają kompletnych danych. Wyróżniamy zwykle dwie metody rozwiązywania tego problemu.

Usuwanie przykładów

Pierwszą z metod jest usuwanie przykładów, w których występują braki, i dalsza praca na pełnych danych. Jest to najprostsza metoda, ale również posiadająca pewne niepożądane konsekwencje. Po pierwsze, eliminując przykłady, pozbywamy się części dostępnej informacji. W rezultacie spodziewamy się, że wytworzony model klasyfikacji będzie działał gorzej. Po drugie, metoda ta sprawia, że późniejsza predykcja przykładów z brakami jest niemożliwa. Może się okazać, że nowe, rzeczywiste przykłady będą posiadały braki i wtedy dalsze działanie na tych przykładach będzie niemożliwe.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Imputacja

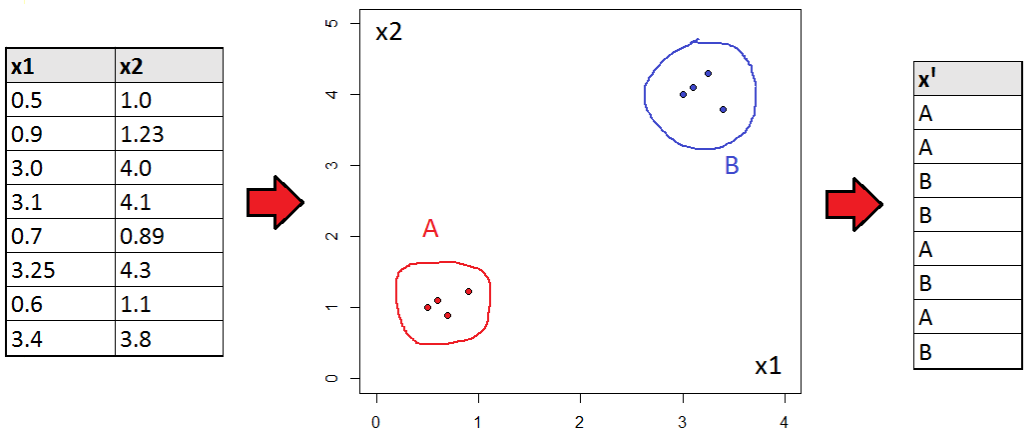
Drugą metodą radzenia sobie z brakującymi wartościami jest *imputacja*. Polega ona na zastępowaniu braków wartościami, stosując do tego pozostałe, dostępne dane. W przypadku atrybutów ciągłych braki można zastępować średnią lub medianą atrybutu. Dane dyskretne można imputować najczęstszą wartością atrybutu (modą). Podejściem nieco bardziej zaawansowanym jest użycie klasyfikatora lub algorytmu regresji w celu przewidywania wartości w pustych miejscach.

2.2.3 Redukcja wymiarów

Ostatnim z rodzajów metod przetwarzania wstępnego jest redukcja wymiarów, czyli zmniejszenie liczby atrybutów. Często bywa tak, że kategoria przykładu jest uzależniona od podzbioru jego atrybutów. Pozostałe atrybuty, nie dość, że nie powinny wnosić nic do procesu klasyfikacji, to jeszcze swoją obecnością wydłużają czas obliczeń i mogą negatywnie wpływać na proces uczenia (stanowić szum). Usuwając te niepotrzebne atrybuty możemy zmniejszyć czas obliczeń oraz pozytywnie wpłynąć na ocenę klasyfikacji.

W celu wybrania właściwych atrybutów do usunięcia możemy usuwać kolejne kombinacje atrybutów, a następnie porównywać oceny klasyfikacji dokonanej na reszcie danych. Metoda ta ma oczywistą wadę – jest bardzo czasochłonna. Inne podejście prezentuje metoda *PCA* (ang. *Principal Component Analysis*) – poszukuje ona takich kombinacji atrybutów, które wiernie oddają wariancję całego zestawu danych [5].

Ostatnim podejściem do redukcji wymiarów jest grupowanie atrybutów. Celem grupowania jest taki podział nieetykietowanych przykładów, który zapewnia ich maksymalne podobieństwo w ramach każdej kategorii i maksymalne zróżnicowanie pomiędzy różnymi kategoriami [6]. W kontekście pracy dyplomowej, grupowanie traktuje wartości n-elementowego podzbioru atrybutów jako punkty w przestrzeni n-wymiarowej. Punkty te są grupowane za pomocą odpowiednich algorytmów uczenia maszynowego (np. *K-średnich*, *DBSCAN*, *Mean Shift*) a następnie zamieniane na etykiety tych grup. W ten sposób z n atrybutów powstaje pojedynczy atrybut etykiet, a $n - 1$ zostaje usuniętych.



Rysunek 2.6: Ilustracja przestawiająca w jaki sposób grupowanie może być użyte do redukcji wymiarów. Dwuwymiarowe dane o atrybutach x_1 i x_2 przeniesiono na płaszczyznę i pogrupowano w dwa zbiory o etykietach *A* i *B*. Następnie oba atrybuty zostały zastąpione pojedynczym atrybutem *x'* zawierającym etykiety przykładów.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

2.2.4 Modele przetwarzania wstępnego

Badając wpływ metod przetwarzania wstępnego na końcową ocenę klasyfikacji należy oddzielić obliczenia na danych trenujących od testowych. Podobnie jak ma to miejsce w przypadku klasyfikacji, metody przetwarzania wstępnego tworzą parametryczne modele bazując na danych trenujących. Przykładowo, modelem binaryzacji atrybutów jest zbiór liczb będących wartościami progów binaryzacji dla każdego atrybutu. Prowadząc badania należy pamiętać o tworzeniu modeli przetwarzania wyłącznie na danych trenujących oraz o stosowaniu tych modeli na osobnych danych testowych.

2.3 Testy statystyczne

Celem mojej pracy dyplomowej jest porównanie jakości klasyfikacji rodzin pewnych algorytmów. Aby porównanie to było wiarygodne i, aby mogło posłużyć dalszym badaniom, powinno ono używać ocen mierzonych na całej populacji, a nie na losowej próbie. Oczywistym jest, że nie dysponujemy informacjami na temat wszystkich pacjentek i możemy użyć jedynie niewielkiego zbioru danych (ok. 1000 przykładów w próbie). W związku z powyższym, aby porównanie było możliwie dokładne, wykonane zostanie z użyciem testu statystycznego.

Testem statystycznym nazywamy formułę matematyczną pozwalającą oszacować prawdopodobieństwo spełnienia pewnej hipotezy statystycznej w populacji na podstawie próby losowej z tej populacji. W przypadku mojej pracy, hipotezą zerową, którą pragniemy obalić, jest jednakowy rozkład ocen klasyfikacji dla wybranych dwóch rodzin metod. Obalenie hipotezy zerowej oznacza różnorodność rozkładów.

Testy statystyczne dzielimy na dwa rodzaje: parametryczne i nieparametryczne. Testy pierwszego rodzaju zakładają pewien ustalony rozkład danych oraz parametry tego rozkładu. Przykładami testów statystycznych są testy *t-Studenta* oraz *korelacja Pearsona*.

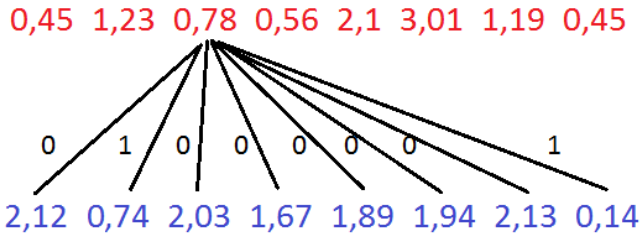
Testy nieparametryczne nie wymagają założeń odnośnie populacji, z której losowana jest próba. Dzięki mniejszej liczbie założeń do spełnienia mogą być szerzej stosowane niż testy parametryczne. Wadą testów nieparametrycznych jest mniej dokładny pomiar oraz mniejsza moc. Mocą testu nazywamy prawdopodobieństwo odrzucenia fałszywej hipotezy zerowej. W przypadku mojej pracy, testy nieparametryczne rzadziej będą wskazywać rzeczywiste różnice w rozkładach ocen pomiędzy rodzinami. Przykładami testów nieparametrycznych są test *Manna-Whitneya-Wilcoxona* oraz test *chi kwadrat*.

Biorąc pod uwagę brak wiedzy a priori na temat rozkładu ocen w rodzinach metod klasyfikacji, do porównywania użyty został test nieparametryczny Manna-Whitneya-Wilcoxona. Test ten polega na policzeniu statystyki U, związanej z liczbą przypadków, gdzie elementy jednej z rodzin mają wartości wyższe niż elementy z drugiej rodziny. Następnie wartość U porównywana jest z wartością graniczną dla ustalonych liczności rodzin oraz poziomu istotności testu. W przypadku bardzo licznych rodzin, statystyka U jest zmienną losową o rozkładzie normalnym. Pozwala to w prosty sposób obliczyć *p-wartość* dla danej obserwacji, a następnie porównać ją z ustalonym poziomem istotności testu.

2.4 Podsumowanie

W tej części pracy przedstawiłem podstawowe informacje na temat przetwarzania wstępnego i klasyfikacji danych. Wyjaśniłem na czym polega klasyfikacja, jak ocenia się klasyfikatory, zwróciłem uwagę na problem nadmiernego dopasowania oraz zaprezentowałem popularne algorytmy klasyfikacji. Oprócz tego wyjaśniłem cel wstępnego przetwarzania danych oraz zaprezentowałem najpopularniejsze metody tej procedury.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00



Rysunek 2.7: Ilustracja prezentująca zasadę działania testu *Manna-Whitneya-Wilcoxona* dla dwóch zbiorów danych - czerwonego i niebieskiego. Każdy element ze zbioru czerwonego porównywany jest z każdym elementem ze zbioru niebieskiego. Porównując wartości zlicza się liczbę "wygranych"dla jednego z dwóch zbiorów.

Następny rozdział poświęcony jest wymaganiom systemu komputerowego służącego do porównywania algorytmów przetwarzania i klasyfikacji.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

3. Założenia i wymagania systemu

Zdefiniowanie wymagań ma bardzo duże znaczenie podczas realizacji projektu informatycznego. Właściwe sformułowanie wymagań uszczegóławia wizję systemu oraz jest źródłem wiedzy o pożądanych możliwościach i cechach rozwiązania. Dodatkowo, prawidłowa identyfikacja wymagań nakreśla cel projektu i, co za tym idzie, pozwala uniknąć potrzeby wprowadzania kosztownych poprawek.

Wymagania dzielimy na funkcjonalne, jakościowe (niefunkcjonalne) oraz wymagania ograniczeń. Do pierwszej grupy zaliczamy wymagania opisujące możliwości, jakie projektowany system daje użytkownikom. Przykładem wymagania funkcjonalnego może być klasyfikowanie danych przy użyciu algorytmu drzew decyzyjnych. Wymagania jakościowe skupiają się na ilościowych regułach oceniania systemu, np. czy baza danych potrafi odzyskać utracone dane w mniej niż jedną sekundę. Ostatnia kategoria wymagań precyzuje granice rozwiązania.

3.1 Wymagania funkcjonalne

Zgodnie z tym, co opisane jest w rozdziale poświęconym celowi pracy, przygotowywany przeze mnie system służy porównywaniu jakości klasyfikacji wielu zbiorów algorytmów. Użytkownik systemu powinien być w stanie wprowadzić informacje o rodzinach algorytmów, które chce porównać pod kątem pewnej miary jakości klasyfikacji. Użytkownik powinien również dostarczyć dane medyczne zapisane w pewnym ustalonym formacie. W efekcie tego system powinien zwrócić dane o cechach szczególnych wprowadzonych rodzin algorytmów (np. wartość średnia oceny). Dodatkowo system powinien uszeregować rodziny algorytmów pod kątem wybranej miary jakości, dając w ten sposób pogląd na wzajemną skuteczność rodzin algorytmów. Informacje o wzajemnej skuteczności rodzin mogą okazać się pomocne przy późniejszym decydowaniu, na której z rodzin należy skoncentrować dalsze badania.

Takie spojrzenie na podstawowe wymagania pozwala wyróżnić trzy podstawowe zestawy funkcjonalności: wprowadzanie danych wejściowych, wykonywanie obliczeń i prezentację wyników.

3.1.1 Wprowadzanie danych wejściowych

Jednym z podstawowych zadań systemu jest wstępne przetwarzanie danych, klasyfikacja oraz obliczenie miary jakości klasyfikacji. Taki ciąg obliczeń w reszcie pracy dla uproszczenia nazywać będę *algorytmem*. Użytkownik, rozpoczynając obliczenia, powinien wprowadzić informacje o rodzinie algorytmów, dla których wyniki oceny chce uzyskać. Każdy algorytm będzie wykonywał pewne operacje na dostarczonym zbiorze danych. Aby ustalić przebieg działania pojedynczego algorytmu należy najpierw przyjrzeć się posiadanym danym biomedycznym.

Opis danych wejściowych

Dostępny zbiór danych zawiera ok. 1000 obserwacji. Każda z nich to zestaw prawdziwych, anonimowych danych otrzymanych w wyniku przebadania pacjentek pod kątem zachorowalności na raka piersi. Każda obserwacja w próbie zawiera sześć atrybutów ciągłych oraz jedną dyskretną kategorię chorobową (choruje/nie choruje).

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Pierwszym atrybutem w każdej obserwacji jest wiek pacjentki. Kolejne pięć to poziomy ekspresji:

1. Swoistego antygenu sterczowego (PSA, z ang. *prostate-specific antigen*)
2. Czynnika martwicy guza alfa (TNFA, z ang. *tumor necrosis factor-alpha*)
3. Interleukiny 6
4. Interleukiny 8
5. Czynnika wzrostu śródbłonka naczyniowego (VEGF, z ang. *vascular endothelial growth factor*)

Zbiór danych zawiera zbliżoną liczbę obserwacji ze stanem chorobowym oraz bez stanu chorobowego.

Dodatkowo, zestaw danych biomedycznych posiada ok. 8% brakujących wartości różnych atrybutów (kategoria nie zawiera braków).

Age	PSA	TNFA	IL6	IL8	VEGF	Index
19	6,249	0,872	0,193	141,201	14,678	0
19	0,338	0,072	0,043	6,399	14,678	0
20	2,603	1,976	0,615	287,745	21,74787	1
20	0,315	1,757	0,268	36,219	29,42391	1
20	0,844	2,094	0,576	9,75	41,076	0
20	1,22	6,313	0,698	80,199	101,2817	1
20	0,895	1,293	0,592	25,399	47,044	0
20	3,287	1,083	0,73	45,051	8,46	1
21	1,212	3,905	4,334	178,719	48,267	0
21	24,355	0,788	1,769	7,199	37,1591	0
21	0,587	0,746	0,637	10,92	1,739	1
22	2,83	1,482	1,541	7,087	27,91919	0
22	2,864	0,978	0,534	5,089	0,567	1
22	0,807	2,014	0,422	17,741	6,857	1

Rysunek 3.1: Fragment zbioru danych dotyczących zachorowalności na raka piersi u kobiet wraz z wyszczególnionymi atrybutami i kategorią (*Index*).

Pojedynczy algorytm

Każdy algorytm składa się z czterech etapów wstępnego przetwarzania oraz z pojedynczego etapu końcowej klasyfikacji:

1. Przetwarzanie wstępne
   a) Metoda imputacji danych brakujących
   b) Wskazanie atrybutów do usunięcia
   c) Wskazanie atrybutów do przeskalowania
   d) Określenie metod grupowania i ich parametrów, wybór atrybutów do zgrupowania
2. Metoda klasyfikacji wraz z parametrami

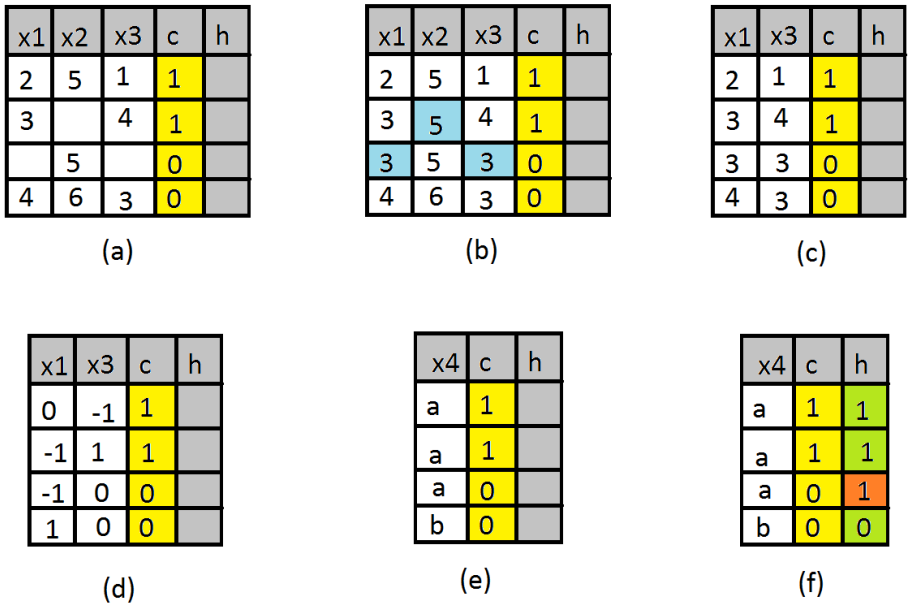
Spośród wyżej wymienionych operacji tylko pierwsza i ostatnia powinny być zawsze określone. Wynika to z faktu, że wiele algorytmów klasyfikacji zakłada obecność wszystkich danych i nie uwzględnia sytuacji, w których pewnych wartości brakuje. Inaczej jest w przypadku pozostałych czynności. Usuwanie, skalowanie oraz grupowanie atrybutów mogą mieć zasadniczy wpływ na przebieg całego algorytmu, ale nie są konieczne. Najprostszy pojedynczy

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

algorytm polegać będzie zatem na imputacji danych wybraną metodą oraz na późniejszej klasyfikacji danych. Najbardziej złożony algorytm zawierać będzie w sobie wszystkie wymienione etapy.

Należy zwrócić uwagę na wymaganie dotyczące grupowania wartości w atrybutach. Aplikacja powinna pozwalać na wykorzystanie wielu różnych algorytmów łączenia jednocześnie. Dla każdego wybranego algorytmu grupowania użytkownik powinien móc wskazać zbiór atrybutów poddanych łączeniu oraz parametry algorytmu.

Fazy algorytmu związane z usuwaniem, skalowaniem i grupowaniem atrybutów wymagają wskazania zbioru atrybutów, na którym konkretna operacja ma zostać wykonana. Liczba możliwych kombinacji atrybutów zależy wykładniczo od liczby atrybutów w zbiorze. Oznacza to, że w ogólnym przypadku rodzina algorytmów nie może zawierać algorytmów ze wszystkimi możliwymi kombinacjami. Wiązałoby się to z nieakceptowalnie długim czasem obliczeń. Biorąc jednak pod uwagę, że dostępne dane zawierają tylko 6 atrybutów, można pozwolić sobie na tworzenie rodzin zawierających wszystkie kombinacje. Dalsze wymagania zakładać będą, że zbiór danych zawierać będzie nie więcej niż 10 atrybutów.



Rysunek 3.2: Przykład działania pojedynczego algorytmu. Dane w postaci początkowej (a), po imputacji medianą (b), po usunięciu atrybutu x_2 (c), po przeskalowaniu do przedziału [-1, 1] (d), po zgrupowaniu atrybutów x_1 i x_3 w nowy atrybut x_4 (e), po predykcji wybraną metodą klasyfikacji (f).

Rodzina algorytmów

System powinien obliczać jakość klasyfikacji rodziny algorytmów zdefiniowanej w pewien sposób przez użytkownika. Projektując metodę opisu rodzin algorytmów należy zachować równowagę pomiędzy jej skrótowością oraz ekspresywnością.

Opis rozwlekły byłby kłopotliwy w użyciu, gdyż wymagałby dużego nakładu pracy, aby opisać rodzinę metod. Dokładna metoda opisu mogłaby wymagać od użytkownika bezpośredniego wprowadzania opisów kombinacji, które aplikacja miałaby obsłużyć. Jednakże przy gigantycznej liczbie wszystkich możliwych wariantów, rozwiązanie to nie znalazłoby zastosowania - opisy rodzin stałyby się po prostu za długie.

Skrajną alternatywą mogłoby być pytanie użytkownika, czy algorytmy w rodzinie powinny uwzględniać określone operacje, np. usuwanie atrybutów lub ich grupowanie. Taki krótki, nieprecyzyjny opis nie pozwalałby na wyrażanie konkretnych rodzajów algorytmów

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

w rodzinie. Odpowiedzi „tak lub nie” byłyby wygodne dla użytkownika, ale odbierałyby jakikolwiek wpływ na kształtowanie rodzin algorytmów.

Całkowita liczba wszystkich możliwych kombinacji metod przetwarzania wstępnego i klasyfikacji zależy wykładniczo od liczby atrybutów danych. Przykładowo, liczba możliwych kombinacji atrybutów, które aplikacja podda skalowaniu, wynosi 2^n, gdzie n oznacza liczbę atrybutów. Aby uniknąć długiego czasu obliczeń związanego z eksplozją wykładniczą, użytkownik powinien być w stanie określić podzbiór wszystkich kombinacji, które program ma przeanalizować. Podzbiór taki można traktować jako podprzestrzeń kombinacji parametrów wszystkich możliwych algorytmów.

Zaproponowane przeze mnie rozwiązanie stanowi kompromis pomiędzy dwoma skrajnymi metodami opisu rodzin. Polega ono na deklaratywnym opisie parametrów algorytmów należących do rodziny.

Na samym początku użytkownik powinien wybrać metody imputacji brakujących wartości atrybutów, na przykład: $\{\textit{mediana}, \textit{wartość średnia}\}$.

W następnej kolejności użytkownik powinien określić zbiór atrybutów R_{attr}, spośród których aplikacja może wybrać te, które zostaną usunięte. Dodatkowo użytkownik powinien określić zbiór liczb $\mathbf{R} = \{r_1, r_2, \ldots, r_n\}$, taki, że każda liczba r_i oznacza liczbę atrybutów, które algorytm będzie mógł usunąć jednocześnie ze zbioru R_{attr}.

Analogicznie powinno przebiegać określanie szczegółów skalowania atrybutów. Użytkownik powinien wybrać zbiór atrybutów N_{attr} oraz wprowadzić zbiór liczb $\mathbf{N}$, którego każdy element oznaczać będzie liczbę atrybutów skalowanych jednocześnie.

Opis podprzestrzeni związanej z grupowaniem okazuje się bardziej skomplikowany. Podobnie jak poprzednio, użytkownik powinien wskazać zbiór atrybutów poddanych grupowaniu Q_{attr}. Następnie użytkownik powinien określić zbiór algorytmów grupowania $\mathbf{Q}$, które mogą być użyte. W kolejnym kroku należy wprowadzić zbiór liczb Q_{pass}. Każdy element zbioru oznaczać będzie liczbę jednoczesnych przebiegów grupowania. W dalszej kolejności odbiorca określi liczbę n_{max} oznaczającą maksymalną liczbę atrybutów, które mogą być łączone przez algorytmy ze zbioru $\mathbf{Q}$. Na samym końcu osoba będzie musiała opisać podzbiór wszystkich kombinacji parametrów, jakie mogą przyjąć wybrane wcześniej algorytmy grupowania. Aby uprościć to zadanie, użytkownik wprowadzi tylko jedną liczbę g, oznaczającą *granularność* podprzestrzeni parametrów.

Wiele algorytmów często przyjmuje parametry początkowe należące do określonych dziedzin. Przykładami takich dziedzin mogą być: wartości „tak” i „nie”, zbiór liczb rzeczywistych od 0.01 do 100. W ogólnym przypadku nie jest możliwe przeglądanie wszystkich kombinacji parametrów dla dowolnego algorytmu – jest ich zbyt dużo lub ich wartości są nieprzeliczalne. Z tego powodu, w mojej pracy przeszukiwanie przestrzeni ograniczam do jej wybranych elementów. Naturalnym podejściem do tego problemu jest wybór takich wartości z każdej dziedziny, które byłby dla niej reprezentatywne.

Granularnością nazywam liczbę, która określa, ile równo oddalonych od siebie wartości należy wybrać z danego przedziału. Parametr ten pozwala próbkować dany przedział liczb z określoną częstością. Umożliwia to mniej lub bardziej dokładne przeglądanie parametrów liczbowych dla algorytmów grupowania i klasyfikacji. W zależności od zastosowania, różnica pomiędzy kolejnymi wartościami z przedziału może być równa na skali liniowej lub logarytmicznej.

Granularność liniową g_l dla zbioru $\{a, \ldots, b\}$ definiuję jako liczność zbioru $\mathbf{L} = \{l_1, l_2, \ldots, l_n\}$ takiego, że:

$$
\begin{aligned}
l_1 &= a \\
l_n &= b \\
l_{i+1} - l_i &= c
\end{aligned}
$$

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

gdzie c jest stałą, a $i \in \{1, \ldots, n-1\}$. Intuicyjnie granularność liniowa to liczba o 1 większa od liczby podziałów zbioru na przedziały o jednakowej długości.

Granularność wykładniczą g_w dla zbioru $\{a, \ldots, b\}$ definiuję jako liczność zbioru $\mathbf{W} = \{w_1, w_2, \ldots, w_n\}$ takiego, że:

$$
\begin{aligned}
w_1 &= a \\
w_n &= b \\
w_{i+1} &= w_i * q
\end{aligned}
$$

gdzie q jest stałą, a $i \in \{1, \ldots, n-1\}$. Posługując się intuicją, granularność wykładniczą można rozumieć jako liczbę o 1 większą od liczby podziałów zbioru na przedziały należące do różnych rzędów wielkości. Zbiory **L** i **W** nazywam *zbiorami granularnymi*.

W projektowanym systemie każdy algorytm grupowania lub klasyfikacji posiada ustaloną przestrzeń dostępnych parametrów. Całkowita przestrzeń parametrów algorytmu P_{tot} składa się z podprzestrzeni P_i każdego pojedynczego parametru i. Zbiór P_{tot} jest zatem iloczynem kartezjańskim zbiorów P_i. Każdy zbiór P_i określony jest przez granice przedziału liczbowego, do którego należy, oraz przez rodzaj granularności (np. liniową). Faktyczna zawartość zbioru P_{tot} znana jest dopiero wówczas, gdy użytkownik ustali wartość granularności. Na podstawie tej informacji generowane są elementy zbiorów P_i, a co za tym idzie, całej przestrzeni P_{tot}.

Po zakończeniu ustalania szczegółów grupowania, użytkownikowi pozostanie ustawienie procesu klasyfikowania danych. Odbiorca aplikacji będzie musiał wskazać algorytmy, których działanie ma zostać sprawdzone, jak również granularność dla tych algorytmów. Jak zatem widać, zbiór parametrów rodziny jednoznacznie wyznacza algorytmy, które do niej należą. Umiejętne dobranie parametrów rodziny umożliwia wygenerowanie mniejszej lub większej liczby algorytmów wewnątrz rodziny.

1. Imputacja:
   a) Podzbiór zbioru metod $\{$*mean*, *median*$\}$, np. $\{$*mean*$\}$
2. Usuwanie i skalowanie atrybutów:
   a) Podzbiór atrybutów $\{$*vegf*, *psa*, *tnfa*, *il6*, *il8*, *age*$\}$, np. $\{$*tnfa*, *il8*$\}$
   b) Zbiór liczności podzbiorów, np. $\{0, 1, 3\}$
3. Grupowanie:
   a) Podzbiór dostępnych algorytmów $\{$*k-means*, *meanshift*$\}$, np. $\{$*k-means*$\}$
   b) Zbiór atrybutów, np. $\{$*vegf*, $\ldots$, *age*$\}$
   c) Zbiór liczb algorytmów używanych jednocześnie, np. $\{2\}$
   d) Maksymalna liczba atrybutów podlegających grupowaniu przez algorytm, np. 2
   e) Granularność, np. 5
4. Klasyfikacji
   a) Podzbiór algorytmów $\{$*bayes*, *randomforest*, *tree*, *svm*$\}$, np. $\{$*tree*$\}$
   b) Granularność, np. 6

Rysunek 3.3: Parametry wchodzące w skład opisu pojedynczej rodziny algorytmów. Opis jest podzielony na części poświęcone imputacji braków, usuwaniu, skalowaniu i grupowaniu atrybutów oraz klasyfikowaniu danych.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

3.1.2 Wykonywanie obliczeń

Część ta koncentruje się na dokładnym opisie obliczeń przeprowadzanych przez system. Jako pierwszy zaprezentowany jest schemat obliczeń, czyli algorytm wg którego system wytwarza wyniki dla danych wejściowych. Następnie omówione jest zagadnienie związane z przechowywaniem wyników obliczeń. W dalszej kolejności omówiona jest kwestia wykonywania obliczeń równolegle. Na samym końcu znajduje się kilka słów na temat podziału danych na zbiory trenujące i testowe.

Schemat obliczeń

Dla ustalonych danych wejściowych X oraz pojedynczego algorytmu A_i ze zbioru algorytmów A, generowanych jest 10 losowych podziałów na zbiory trenujące $X_s = X_{s1}, \ldots, X_{s10}$ oraz testowe $X_t = X_{t1}, \ldots, X_{t10}$ (walidacja krzyżowa).

W pierwszej kolejności dane są wstępnie przetwarzane. Na danych trenujących X_{si} tworzony jest model imputacji m_p zgodny z wybraną metodą algorytmu A_i. Następnie brakujące dane trenujące są zastępowane wartościami zwróconymi przez ten model. Jeżeli algorytm uwzględnia usuwanie atrybutów, czynność ta jest wykonywana zaraz po tym. Kolejnym krokiem jest przeskalowanie wybranych atrybutów. W tym celu tworzony jest model skali atrybutów m_s, który następnie użyty jest to transformacji wybranych atrybutów zbioru trenującego. Ostatnim krokiem przetwarzania wstępnego jest grupowanie. Opierając się na przekształconych danych trenujących, system generuje model grupowania m_g odpowiadający algorytmom grupowania zawartym w opisie algorytmu A_i. Model grupowania użyty jest następnie na danych trenujących doprowadzając je do ich ostatecznej formy.

Przetwarzanie wstępne danych testowych X_{ti} przebiega podobnie, z tą różnicą, że do imputacji, skalowania oraz grupowania używa się gotowych modeli m_p, m_s i m_g.

Po zakończeniu fazy przetwarzania nadchodzi kolej na ocenę klasyfikacji. Na przetworzonym zbiorze trenującym wytworzony jest model klasyfikacji m_k zgodnie z wybranym algorytmem klasyfikacji zawartym w opisie A_i. Model m_k użyty jest do predykcji kategorii na przetworzonym zbiorze testowym. W oparciu na wartościach komórek z macierzy pomyłek obliczane są miary jakości oceny - precyzja, wrażliwość oraz F1.

Powyższe obliczenia powtarzane są dla każdego z 10 podziałów na zbiory trenujące i testowe. Oznacza to, że wynikiem dla każdego algorytmu A_i ze zbioru A jest 30 liczb - 10 trójek miar jakości oceny.

Wyżej opisane obliczenia wykonywane są dla każdego algorytmu w rodzinie. Po zakończeniu obliczeń oceny poszczególnych algorytmów są łączone w jeden zbiór i przypisywane do rodziny algorytmów.

Posiadając wyniki obliczeń dla każdej z rodzin, można przejść do porównania rodzin. Najpierw dla każdej pary rodzin należy ocenić, czy wyniki w rodzinach różnią się od siebie. W tym celu używa się testu Manna-Whitneya-Wilcoxona. Jeżeli wyniki rodzin różnią się od siebie, wygrywa ta z nich, której różnica średniej i odchylenia standardowego jest większa. Dla każdej rodziny i należy policzyć wartość t_i oznaczającą z iloma pozostałymi rodzinami „wygrała”. Na końcu należy uszeregować rodziny używając wartości t_i.

Całość obliczeń w dużym skrócie prezentuje algorytm 1.

Prowadzenie obliczeń

Wykonywanie obliczeń opisanych w poprzednim podpunkcie może być bardzo czasochłonne. Z tego powodu system powinien dawać możliwość wstrzymania obliczeń w dowolnym momencie oraz kontynuowania ich po dowolnie długiej przerwie. Dodatkowo, system powinien

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Wejście: zbiór rodzin algorytmów A
Wyjście: ranking R
Dla każdego $A_i \in A$
  Dla każdego $a_j \in A_i$
    Oblicz miary oceny klasyfikacji M_{a_j};
  Koniec
  $Y_i = \bigcup M_{a_j}$;
  Porównaj oceny Y_i z ocenami poprzednich rodzin $Y_k, k \in [1, i)$;
Koniec
Utwórz ranking rodzin R;
Zwróć R

Algorithm 1: Uproszczony schemat obliczeń

pokazywać użytkownikowi uproszczoną informację o postępie obliczeń oraz o przewidywanym czasie zakończenia.

Przechowywanie wyników

Z wymagania dotyczącego wstrzymywania i wznawiania obliczeń wynika konieczność przechowywania informacji o cząstkowych rezultatach. Co więcej, system powinien zapamiętywać wyniki obliczeń dla rodzin algorytmów, aby późniejsze zapytania o te rodziny zwracały informacje błyskawicznie.

Rozproszenie obliczeń

Jak łatwo zauważyć, możliwe jest zdefiniowanie rodzin algorytmów, których liczność jest bardzo duża (zależna wykładniczo od niektórych parametrów). Oznacza to, że badania przeprowadzanie na takich rodzinach będą bardzo czasochłonne. W celu skrócenia czasu, obliczenia można wykonywać równolegle na wielu maszynach. Projektowany system powinien umożliwiać wykonywanie obliczeń na pojedynczej maszynie (np. na laptopie) lub na grupie maszyn połączonych siecią komputerową.

Rozpraszając obliczenia na wiele maszyn, należy mieć na uwadze wpływ opóźnień w sieci na wydajność przebiegu badań. Komunikacja sieciowa jest wolniejszą metodą wymiany informacji niż np. komunikacja międzywątkowa lub międzyprocesowa. Należy również zdawać sobie sprawę z problemu związanego z zawodnością zdalnych komputerów. Z losowych przyczyn mogą się one zawiesić, wyłączyć, wprowadzając zakłócenia do całego procesu obliczeń.

Ustalony podział zbioru danych

Porównując oceny jakości klasyfikacji dla różnych rodzin algorytmów należy zadbać o to, aby podziały na zbiory trenujące i testowe były jednakowe dla każdego algorytmu. Tylko wtedy mamy pewność że miary jakości klasyfikacji odnoszą się do efektów predykcji klasyfikatora na tych samych zbiorach danych. Warunek ten powinien być spełniony również wtedy, gdy obliczenia robione są na wielu niezależnych maszynach oraz, gdy raz przerwane obliczenia zostały wznowione.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

3.1.3 Prezentacja wyników

Ostatnią częścią poświęconą wymaganiom funkcjonalnym są wymagania dotyczące prezentacji wyników obliczeń. Istotnym jest, aby raz policzone rozkłady miar można było łatwo zobrazować i przeanalizować.

Informacje o rodzinach algorytmów

System komputerowy powinien pozwalać na podgląd obliczonych miar jakości klasyfikacji dla rodzin algorytmów zdefiniowanych przez użytkownika. System powinien dawać możliwość podglądu podstawowych statystyk dla każdej z rodzin algorytmów. Do wartości tych powinny zaliczać się średnia ocen oraz odchylenie standardowe dla każdej z miar (precyzja, wrażliwość, F1) z osobna. Dodatkowo, aplikacja musi umożliwiać graficzny podgląd histogramów rozkładów ocen wraz z uwzględnieniem zapisu wykresów do plików. Warto również wspomnieć o tym, że dla każdej rodziny algorytmów system powinien wyświetlić prosty i zrozumiały opis, pozwalający zrozumieć jak rodzina została zdefiniowana (metoda imputacji, atrybuty do usunięcia itp).

Ranking rodzin algorytmów

Celem projektowanego systemu komputerowego jest porównywanie jakości predykcji rodzin algorytmów. Porównanie to powinno być zrealizowane w postaci rankingu tych rodzin. Jak zostało to opisane w sekcji 3.1.2, porządek w rankingu powinien być oparty na liczbie „zwycięstw” pomiędzy rodzinami algorytmów. Rodziny powinny być przedstawione od najlepszej do najgorszej wg kryterium wybranej miary oceny (np. precyzji). W celu uproszczenia porównywania rodzin między sobą, każda pozycja w rankingu powinna zawierać:

1. Opis rodziny algorytmów wynikający z parametrów rodziny
2. Informacje o miejscu w rankingach tworzonych wg pozostałych miar ocen
3. Wartości średnie miar oceny klasyfikacji
4. Informacje o rodzinach z którymi zwyciężyła (dla każdej z miar ocen)

3.2 Wymagania niefunkcjonalne

W związku z tym, że praca dyplomowa ma głównie charakter badawczy, etap analizy wymagań skupia się na wymaganiach funkcjonalnych. Mimo to, opis wymagań niefunkcjonalnych nie może być pominięty, ponieważ mogłoby to doprowadzić do sytuacji, w której działanie systemu stwarzałoby problemy utrudniające prowadzenie badań.

3.2.1 Rozmiar przechowywanych danych

Projektowany system komputerowy powinien umożliwiać zapis informacji o wynikach dla 1000 rodzin algorytmów. Każda rodzina może posiadać do 1000000 wyników dla każdej z miar oceny klasyfikacji. System powinien być w stanie wygenerować ranking dla rodzin z takimi ograniczeniami.

3.2.2 Dane wejściowe

Dostępne dane medyczne znajdują się w pliku MS Excel, a wyszczególniona kolumna z kategorią chorobową nosi nazwę *Index*. Projektowane narzędzie powinno być dostosowane do posiadanych danych jak również umożliwiać pracę na innych zbiorach zapisanych w podobny

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

sposób. Z tego powodu system powinien radzić sobie z odczytem danych zgromadzonych w popularnych formatach plików, np. MS Excel lub CSV (*Comma Separated Values*). Z racji tego, że wprowadzane dane poddawane są algorytmom klasyfikacji, plik powinien posiadać wyszczególnioną kolumnę zawierającą informacje o kategorii obserwacji. Podobnie jak ma to miejsce w podstawowym zestawie danych, nagłówek tej kolumny powinien nosić nazwę *Index*.

3.2.3 Algorytmy uczenia maszynowego

System powinien stwarzać możliwość porównania rodzin używających popularnych algorytmów klasyfikacji i grupowania. Aplikacja powinna udostępniać co najmniej 4 algorytmy klasyfikacji danych oraz co najmniej 2 algorytmy grupowania atrybutów.

3.3 Wymagania systemowe

Biorąc pod uwagę różnorodność popularnie używanych systemów operacyjnych, projektowany system nie powinien wymuszać na użytkowniku żadnej konkretnej platformy. Projektowane rozwiązanie powinno działać równie dobrze zarówno na systemach z rodziny MS Windows, jak i Linux. Jednocześnie aplikacja powinna umożliwiać przetwarzanie wstępne oraz klasyfikowanie danych w prosty i intuicyjny sposób. W tym celu system powinien oferować nieskomplikowany, konsolowy interfejs użytkownika. Interfejs musi pozwalać na wprowadzanie danych do przetwarzania oraz na łatwe ustawianie parametrów opisanych w części 3.1.1.

3.4 Wymagania ograniczeń

Projektując oprogramowanie, należy uwzględnić obecność wielu czynników ograniczających. Do czynników ograniczających można zaliczyć ograniczenia finansowe, związane z określonym budżetem przeznaczonym na realizację projektu lub jego określonej fazy. Innym rodzajem ograniczenia jest specyfika środowiska uruchomieniowego. Aplikacje wykonujące przetwarzanie rozproszone mogą wymagać określonej architektury sieci, podczas gdy niektóre systemy wbudowane potrafią działać poprawnie tylko na dedykowanych platformach sprzętowych. Ograniczenia nałożone na projekt mają często bezpośredni wpływ na architekturę końcowego rozwiązania i dlatego są równie istotne jak wymagania funkcjonalne i jakościowe.

Podstawowym ograniczeniem w mojej pracy środowisko uruchomieniowe – jest nim pojedynczy komputer PC. Poprawne działanie aplikacji nie wymaga pracy wielu komputerów, wystarczyć powinna jedna maszyna. System nie powinien wymagać dostępu do Internetu aby działał poprawnie. Tak przedstawione wymaganie nie stoi w sprzeczności z możliwością równoległego liczenia na wielu komputerach, o którym mowa w punkcie 3.1.2. Oznacza ono po prostu, że użytkownik będzie mógł korzystać z systemu np. na pojedynczym laptopie, będąc w podróży.

3.5 Podsumowanie

Tabela 3.1 stanowi zestawienie wymagań projektowych opisanych w tym rozdziale.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Rodzaj wymagania	Opis wymagania
Funkcjonalne	Obliczanie miar jakości klasyfikacji dla zdefiniowanych przez użytkownika rodzin algorytmów metodą walidacji krzyżowej.
Funkcjonalne	Definiowanie rodzin algorytmów poprzez wprowadzanie odpowiednich parametrów.
Funkcjonalne	Każdy algorytm zawiera etapy imputacji i klasyfikacji. Opcjonalne etapy to usuwanie, skalowanie i grupowanie atrybutów.
Funkcjonalne	Porównywanie zbiorów ocen testem Manna Whitneya Wilcoxona.
Funkcjonalne	Możliwość wstrzymywania obliczeń.
Funkcjonalne	Możliwość uruchamiania obliczeń od momentu wstrzymania.
Funkcjonalne	Zapis rezultatów obliczeń (ocen) w bazie danych.
Funkcjonalne	Skrócenie czasu obliczeń poprzez wykonywanie ich na wielu maszynach jednocześnie.
Funkcjonalne	Ustalony podział na zbiory trenujące i testowe.
Funkcjonalne	Informacja o postępie obliczeń.
Funkcjonalne	Informacja o przewidywanym czasie zakończenia obliczeń.
Funkcjonalne	Podgląd średniej i odchylenia standardowego dla każdej policzonej rodziny algorytmów.
Funkcjonalne	Generowanie histogramu miar jakości dla każdej rodziny algorytmów.
Funkcjonalne	Wyświetlanie czytelnych i zrozumiałych opisów rodzin.
Funkcjonalne	Tworzenie rankingu rodzin.
Niefunkcjonalne	Zapis do 1000 rodzin algorytmów w bazie danych.
Niefunkcjonalne	Zapis do 1000000 ocen dla każdej miary dla każdej rodziny algorytmów.
Niefunkcjonalne	Odczyt danych posiadających wyszczególnioną kolumnę kategorii.
Niefunkcjonalne	Obsługa formatu MS Excel.
Niefunkcjonalne	Wybór spośród co najmniej 4 algorytmów klasyfikacji.
Niefunkcjonalne	Wybór spośród co najmniej 2 algorytmów grupowania.
Systemowe	Praca pod systemami operacyjnymi z rodziny MS Windows i Linux.
Systemowe	Prosty i intuicyjny interfejs konsolowy.
Ograniczeń	Aplikacja nie wymaga dostępu do Internetu.
Ograniczeń	Aplikacja działa na pojedynczym komputerze PC.

Tablica 3.1: Wymagania projektowe

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

4. Projekt systemu komputerowego

Rozdział ten skupia się nad projektem systemu komputerowego umożliwiającego porównywanie jakości klasyfikacji dla różnych rodzin algorytmów. Aplikacja jest podzielona na oddzielne, współpracujące ze sobą, elementy. Zaproponowany przeze mnie system komputerowy składa się z gotowych komponentów, o którym mowa w części 4.1, oraz z fragmentów, które należy, chociaż w pewnej części, zaimplementować samodzielnie. W skład tych części wchodzi przede wszystkim biblioteka obliczeniowa *mltool*, program wykonujący obliczenia, program nadzorujący przeprowadzanie obliczeń oraz programy prezentujące ostateczne wyniki.

4.1 Architektura systemu

Dzięki rozdzieleniu całości na powiązane, lecz odrębne, moduły, prace nad wybranym fragmentem systemu można prowadzić w oderwaniu od reszty. Dodatkowo, luźno powiązana architektura umożliwia testowanie każdego podsystemu oddzielnie. Upraszcza to znacząco testowanie całej aplikacji.

4.1.1 Przegląd rozwiązań

System komputerowy powinien być w stanie prowadzić obliczenia na wielu maszynach w sposób równoległy (patrz: 3.1.2). Jedną z metod rozwiązania takiego problemu mogłoby być użycie platformy *Apache Hadoop*. System ten jest otwartą implementacją paradygmatu *MapReduce* firmy Google. Platforma Hadoop umożliwia równoległe przetwarzanie danych tekstowych na wielu maszynach. System ten zapewnia ciągłą pracę w momencie awarii niektórych maszyn wykonujących obliczenia. Z punktu widzenia pracy dyplomowej, system Apache Hadoop posiada jednak pewne wady.

Po pierwsze, jest on zaprojektowany do równoległego przetwarzania danych tekstowych (np. zliczania częstości występowania słów w tekście). Z drugiej strony, obliczenia w projektowanym systemie polegać będą na równoległym obliczaniu miar jakości klasyfikacji algorytmów. Aby pogodzić wymagania systemu z funkcjonalnością platformy Hadoop należałoby przygotować oprogramowanie reprezentujące oceniane algorytmy w postaci tekstowej zrozumiałej przez system Hadoop.

Po drugie, platforma Hadoop jest wysoko rozwiniętym i skomplikowanym systemem. Uruchomienie klastra obliczeniowego wiązałoby się z długim czasem poświęconym na dostrajanie konfiguracji systemu.

Alternatywnym rozwiązaniem jest użycie rozproszonego systemu kolejkowego tj. *Apache ActiveMQ*, *RabbitMQ* lub *ZeroMQ*. Podstawowym zadaniem systemów kolejkowych jest kolejkowanie zleceń wykonania pewnych czynności (np. obliczeń) i późniejsze rozsyłanie ich pomiędzy procesy realizujące te zlecenia. Systemy kolejkowe pozwalają na elastyczne definiowanie przepływu danych pomiędzy różnymi kolejkami.

W odróżnieniu od platformy Hadoop, procesy wykonujące zlecenia odbierają je w jako komunikaty, bardzo często dowolnej postaci. Systemy kolejkowe są również z reguły proste do opanowania i nie wymagają skomplikowanej konfiguracji od użytkownika. Wadą systemów kolejkowych jest brak odporności na awarie procesów-odbiorców oraz maszyn, na których procesy te są uruchomione. Mając na uwadze, że czas obliczeń w projektowanym systemie nie

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

powinien przekraczać kilku dni, prawdopodobieństwo awarii i ryzyko z nim związane jest pomijalne. W związku z tym, wybrano architekturę opartą na systemie kolejkowym.

4.1.2 Opis architektury

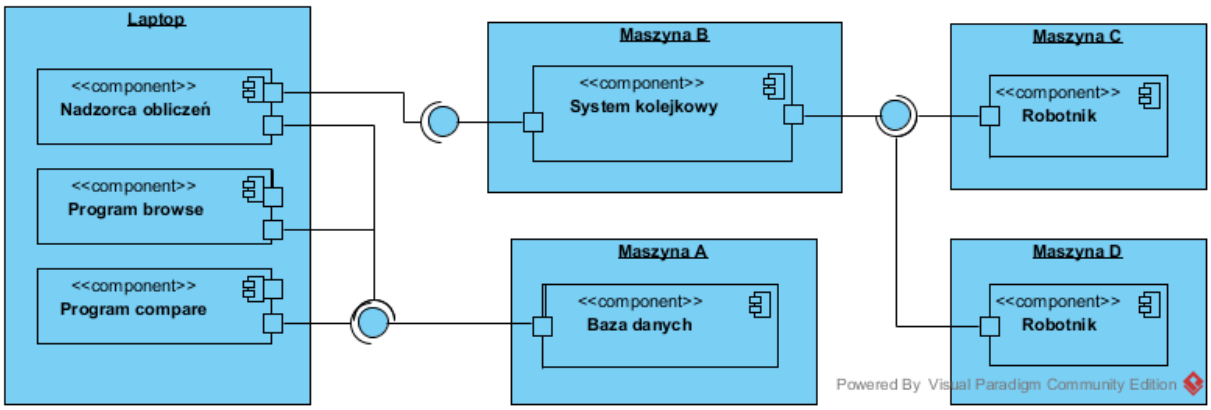
System komputerowy składa się z *procesów-robotników* wykonujących czasochłonne obliczenia (związane z oceną klasyfikacji) oraz pojedynczego *procesu-nadzorcy* rozdzielającego zadania do wykonania. Dodatkowym zadaniem nadzorcy jest odbieranie cząstkowych wyników od każdego robotnika i umieszczanie ich w bazie danych. Komunikacja pomiędzy nadzorcą a robotnikami jest zapewniona poprzez zastosowanie systemu kolejkowego RabbitMQ. System ten został wybrany ze względu na cenę (darmowy), prostotę instalacji, wysoką wydajność oraz bardzo dobre wsparcie w wielu językach programowania (np. w języku Python).

Dane przechowywane są w wolno dostępnej bazie *PostgreSQL*. Wybierając bazę danych kierowałem się przede wszystkim ceną (baza darmowa), łatwością instalacji i konfiguracji oraz wydajnością. Na wybór bazy miały wpływ również moja znajomość języka SQL oraz obecność wielu narzędzi mapowania obiektowo-relacyjnego, uproszczających pracę z danymi.

Proces nadzorujący oraz procesy wykonujące obliczenia używają wspólnej biblioteki obliczeniowej *mltool*.

Programy służące do prezentacji wyników obliczeń dla każdej rodziny algorytmów oraz do policzenia rankingu rodzin pobierają dane z bazy PostgreSQL oraz również używają biblioteki *mltool*.

Poglądowy schemat architektury systemu komputerowego umieszczony został na rysunku 4.1.



Rysunek 4.1: Schemat architektury systemu komputerowego

4.2 Biblioteka obliczeniowa mltool

4.2.1 Struktura biblioteki

Biblioteka obliczeniowa składa się z czterech części. Pierwszą z nich jest zbiór klas odpowiedzialnych za odczyt i dostęp do danych biomedycznych. W skład drugiej części biblioteki wchodzą klasy pozwalające na manipulowanie danymi (np. na skalowanie atrybutów) oraz na ich klasyfikację. Kolejną częścią biblioteki są klasy związane z przetwarzaniem wstępnym i klasyfikacją danych w sposób zdefiniowany przez użytkownika. Stanowią one sparametryzowany opis operacji, jakie powinny być wykonane przez proces liczący (robotnika). Ostatnim fragmentem biblioteki obliczeniowej są klasy związane z generowaniem algorytmów przetwarzania i klasyfikacji odpowiadających opisowi danej rodziny algorytmów.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

4.2.2 Odczyt danych

W części tej znajdują się opisy klas odpowiedzialnych za odczyt i właściwą reprezentację danych.

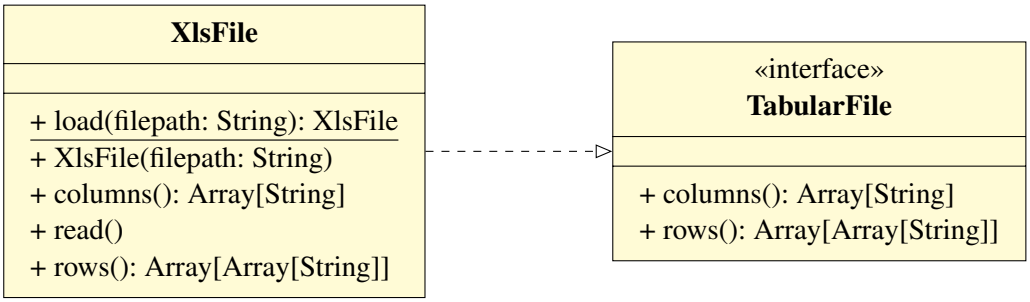
Interfejs TabularFile

Zadaniem interfejsu *TabularFile* jest zapewnienie warstwy abstrakcji pomiędzy odczytem danych z pliku, a interpretacją danych w nim zgromadzonych. Jeżeli projekt będzie rozwijany w przyszłości, może okazać się, że pojawi się potrzeba odczytu plików różnych rodzajów (np. CSV lub formaty binarne). Obecność ujednoliconego interfejsu uprości w takim przypadku rozwój systemu.

Interfejs posiada tylko dwie metody: *columns()* oraz *rows()*. Pierwsza z nich zwraca tablicę nazw kolumn w pliku. Druga zwraca tekstową tablicę dwuwymiarową odpowiadającą kolejnym wierszom danych w pliku.

Klasa XlsFile

W tym momencie przewidywana jest tylko jedna implementacja interfejsu *TabularFile* - klasa *XlsFile*. Klasa ta pozwala na odczyt danych zgromadzonych w postaci tabeli z pliku MS Excel. Diagramy interfejsu *TabularFile* oraz klasy *XlsFile* prezentuje rysunek 4.2.



Rysunek 4.2: Diagram UML interfejsu *TabularFile* i klasy *XlsFile*

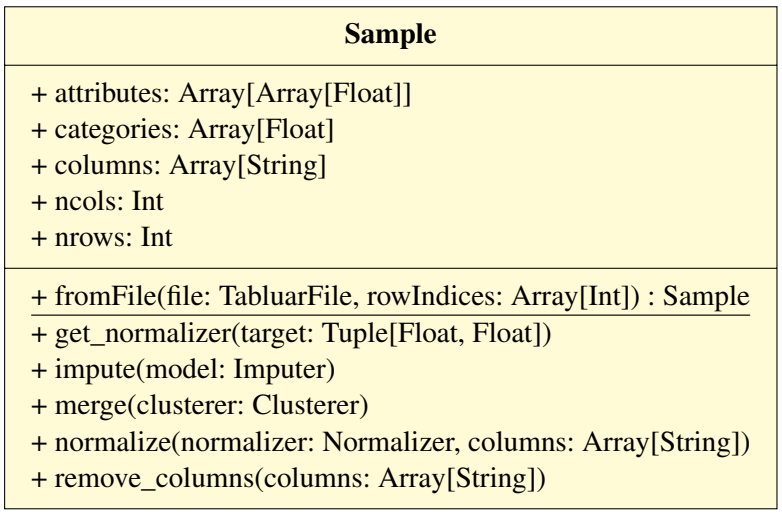
Konstruktor klasy *XlsFile* przyjmuje ścieżkę do pliku z danymi. Metoda *read()* ładuje zawartość pliku do obiektu klasy. Statyczna metoda *load()* zwraca załadowany obiekt klasy *XlsFile*. Jej jedynym argumentem jest ścieżka do pliku. Metody *columns()* oraz *rows()* to implementacje metod interfejsu *TabularFile*.

Klasa Sample

Klasa *Sample* stanowi reprezentację próbki danych zawartych w pliku dostarczonym przez użytkownika. Klasa ta zawiera pola odpowiadające cechom próbki (liczba wierszy, liczba i nazwy atrybutów) jak również udostępnia metody manipulujące danymi. Instancje obiektów klasy *Sample* przechowują wyłącznie atrybuty o ustalonych wartościach numerycznych. Wartości brakujące reprezentowane są prze numeryczną wartość *NaN* (ang. *Not A Number*). Pola i metody klasy prezentuje diagram 4.3.

Pole *attributes* jest tablicą dwuwymiarową przechowującą informacje o wartościach numerycznych poszczególnych atrybutów. Pole *categories* to tablica wartości numerycznych odpowiadających kategoriom kolejnych przykładów (0 - stan chorobowy, 1 - brak stanu chorobowego). Pole *columns* to tablica nazw kolejnych atrybutów. Pola *ncols* i *nrows* przechowują informacje o liczbie kolumn i liczbie przykładów w próbce danych.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00



Rysunek 4.3: Diagram UML klasy *Sample*

Statyczna funkcja *fromFile()* służy do utworzenia obiektu klasy *Sample* na podstawie istniejącego obiektu klasy t_f implementującej interfejs *TabularFile*. Funkcja ta jako argumenty przyjmuje obiekt t_f oraz, opcjonalnie, tablicę indeksów wierszy, które należy wybrać z obiektu t_f.

Metoda *get_normalizer()* zwraca obiekt klasy Normalizer, stanowiący model skalowania atrybutów w zbiorze danych (patrz: 4.2.3).

Zadaniem metody *impute()* jest imputacja brakujących wartości atrybutów w próbie danych zawartej w obiekcie klasy *Sample*. Metoda ta przyjmuje pojedynczy argument - obiekt klasy *Imputer*, będący model imputacji danych.

Celem metody *merge()* jest zgrupowanie wybranych atrybutów ze zbioru danych wg reguł ustalonych w obiekcie *Clusterer* (atrybuty do pogrupowania, algorytmy grupowania oraz parametry tych algorytmów). Parametrem metody jest obiekt klasy *Clusterer*.

Metoda *normalize()* ma za zadanie przeskalować wybrane atrybuty ze zbioru danych. Parametrami metody są obiekt klasy *Normalizer* oraz tablica nazw atrybutów, które mają zostać przeskalowane.

Ostatnią metodą klasy *Sample* jest *remove_columns()*. Zgodnie z tym, co wskazuje nazwa, metoda usuwa ze zbioru danych wybrane atrybuty. Argumentem metody jest tablica nazw atrybutów do usunięcia.

4.2.3 Modele przetwarzania i klasyfikacji

Efektem działania algorytmu przetwarzania wstępnego i klasyfikacji jest wytworzenie modelu, który w pewien sposób opisuje wiedzę zawartą w danych. Model ten wytwarzany jest przy użyciu danych trenujących i wykorzystywany jest do predykcji kategorii, nieznanych wcześniej, danych testowych.

Model powinien uwzględniać wszystkie etapy algorytmu, a fazy przetwarzania i klasyfikacji. Model przetwarzania musi przechowywać informacje o cechach danych istotnych z punktu widzenia transformacji danych. Model taki powinien opisywać cechy szczególne danych pod kątem kolejnych operacji, czyli imputacji brakujących wartości, usuwania atrybutów, skalowania atrybutów i ich późniejszego grupowania.

Model klasyfikacji powinien tworzyć reprezentację wiedzy zawartej w danych w taki sposób, aby możliwe było wykorzystanie jej do późniejszej predykcji nowych danych.

Z punktu widzenia projektu aplikacji, modele powinny udostępniać zbliżony interfejs programistyczny. Interfejs ten w każdym przypadku powinien zawierać przynajmniej jedną metodę

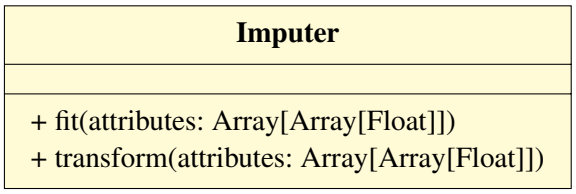
Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

o nazwie *fit*. Metoda ta jest odpowiedzialna za wygenerowanie modelu z dostępnych danych. Dodatkowo, modele przetwarzania muszą posiadać metodę *transform* służącą do transformacji pojedynczego wiersza danych zgodnie z uzyskanym modelem. Modele klasyfikacji powinny posiadać metodę *predict*, której użycie zwracałoby przewidywaną kategorię przykładu.

Klasa Imputer

Klasa *Imputer* służy do imputacji brakujących danych atrybutów. Konstruktor klasy może przyjmować pojedynczą wartość oznaczającą metodę imputacji. Poprawnymi wartościami parametru są *mean* oraz *median*. Metoda *fit* przyjmuje również tylko jeden parametr - dwuwymiarową tablicę zawierającą wartości atrybutów (w tym braki oznaczone jako *NaN*). Metoda transform pobiera jeden argument będący dwuwymiarową tablicę wartości atrybutów. Po wywołaniu metody wartości *NaN* występujące w tablicy zostają zastąpione przez średnią lub medianę odpowiedniego atrybutu.

Rysunek 4.4 ilustruje wygląd klasy *Imputer*.



Rysunek 4.4: Diagram UML klasy *Imputer*

Klasa Normalizer

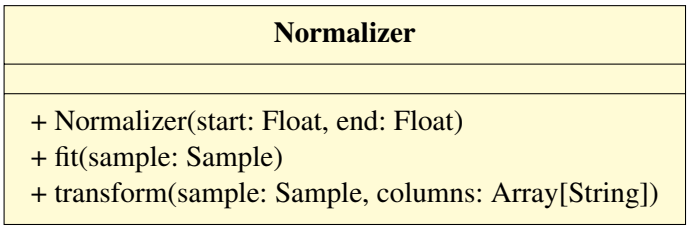
Zadaniem klasy *Normalizer* jest przechowywanie informacji o numerycznym zakresie wartości poszczególnych atrybutów. Po wygenerowaniu modelu, klasa *Normalizer* może być użyta do przeskalowania wartości atrybutu do ustalonego zakresu.

Parametrem konstruktora klasy jest para liczb odpowiadająca zakresowi liczbowemu z, do którego powinny być przeskalowane napływające dane.

Klasa posiada dwie metody: *fit* oraz *transform*. Metoda *fit* posiada jeden parametr - obiekt klasy *Sample*. Efektem wywołania metody *fit* jest wytworzenie modelu zakresu atrybutów danych.

Zadaniem metody *transform* jest przeskalowanie danych wejściowych do wybranego zakresu liczbowego z. Metoda posiada dwa argumenty. Pierwszy z nich to obiekt klasy *Sample*, którego atrybuty mają zostać przeskalowane. Drugi argument to tablica nazw atrybutów, które mają być poddane skalowaniu.

Klasa *Normalizer* jest przedstawiona na rysunku 4.5.



Rysunek 4.5: Diagram UML klasy *Normalizer*

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Klasa Clusterer

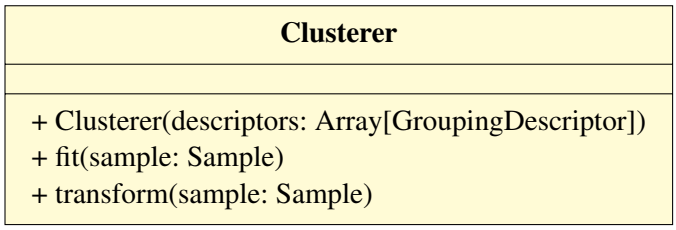
Celem klasy *Clusterer* jest wytworzenie modelu grupowania danych, zgodnego z ustawieniami użytkownika, oraz późniejsze grupowanie nowych danych przy użyciu tego modelu.

Konstruktor klasy *Clusterer* pobiera jeden argument będący tablicą obiektów klasy *GroupingDescriptor*. Każdy obiekt klasy posiada informację o szczegółach grupowania podzbioru atrybutów. Szczegółowy opis klasy *GroupingDescriptor* znajduje się w części 4.2.4.

Klasa posiada dwie metody: *fit* oraz *transform*. Metoda fit posiada jeden argument - obiekt klasy *Sample*. Po wywołaniu metody fit obiekt klasy *Clusterer* tworzy wewnętrzny model wiedzy dotyczącej grupowania atrybutów w zbiorze danych.

Metoda transform posiada również tylko jeden parametr, będący ponownie obiektem klasy *Sample*. Efektem wywołania metody *transform* jest pogrupowanie atrybutów w zbiorze danych zgodnie z modelem wytworzonym metodą *fit*.

Na rysunku 4.6 przedstawiono metody klasy *Clusterer*.



Rysunek 4.6: Diagram UML klasy *Clusterer*

Usuwanie atrybutów

W przypadku usuwania atrybutów ze zbioru danych nie jest potrzeby żaden specjalny model wiedzy. Aby w jednakowy sposób pozbyć się atrybutów w zbiorze trenującym i testowym wystarczy utworzyć tablicę nazw atrybutów, które należy usunąć.

Interfejs Classifier

Interfejs *Classifier* zapewnia możliwość nauki modelu klasyfikacji oraz predykcji kategorii nowych przykładów. Interfejs ma dwie metody: *fit()* i *predict()*.

Metoda *fit()* służy do nauki modelu danych. Metoda ta przyjmuje pojedynczy parametr będący tablicą dwuwymiarową zawierającą wartości atrybutów zbioru danych. Po wywołaniu tej metody obiekt implementujący interfejs *Classifier* wytwarza swój własny model klasyfikacji.

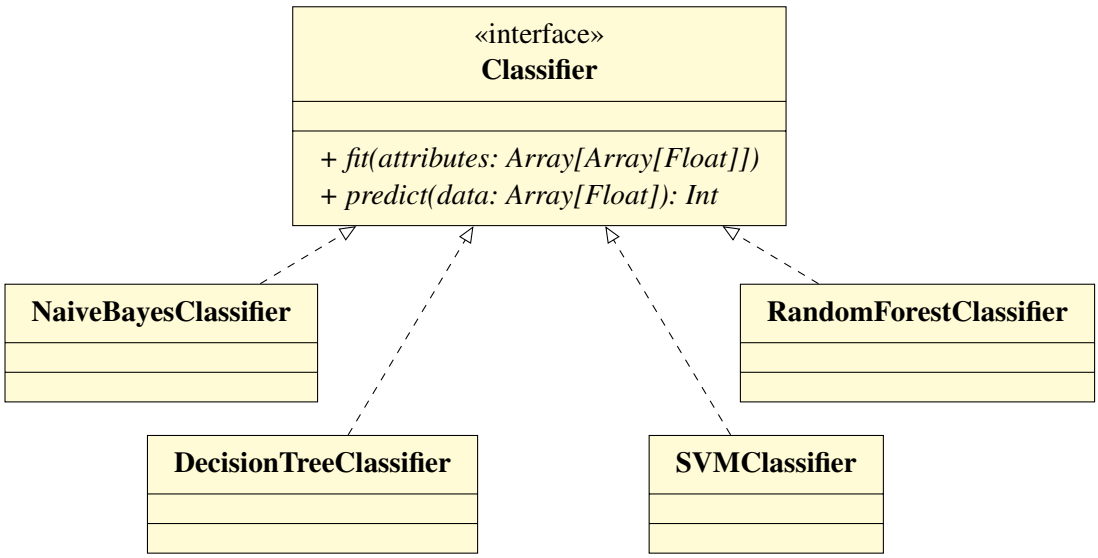
Zadaniem metody *predict()* jest predykcja kategorii przykładu ze zbioru danych. Metoda przyjmuje tablicę wartości odpowiadającą pojedynczemu przykładowi ze zbioru danych. Wynikiem wywołania metody jest wartość predykcji - przewidywana kategoria przykładu.

Interfejs *Classifier* jest zaimplementowany przez cztery klasy: *NaiveBayesClassifier*, *DecisionTreeClassifier*, *SVMClassifier* oraz *RandomForestClassifier*. Odpowiadają one algorytmom klasyfikacji naiwny klasyfikator bayesowski, drzewo decyzyjne, maszyna wektorów nośnych i las losowy. Całość przedstawia rysunek 4.7.

4.2.4 Deskryptory operacji

Projektując bibliotekę postanowiłem oddzielić kod związany z wyborem parametrów przetwarzania od kodu wykonującego to przetwarzanie. W tym celu zaprojektowałem specjalne klasy, tzw. *deskryptory*. Klasy te czerpią ze wzorca projektowego *Polecenie* [7], stanowiąc opisy

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00



Rysunek 4.7: Diagram UML interfejsu *Classifier*

operacji, które mają zostać wykonane. Każdy z deskryptorów dotyczy szczególnej części przetwarzania i posiada metody wykonujące daną operację oraz metody sprawdzające poprawność opisu operacji.

Użycie deskryptorów posiada wiele zalet. Po pierwsze, logika związana z definiowaniem kolejnych kroków algorytmu jest odseparowana do właściwego algorytmu. Efektem tego jest prostszy i czytelniejszy kod. Po drugie, każdy deskryptor posiada metody umożliwiające sprawdzenie, czy wprowadzony opis operacji jest poprawny. Pomaga to w zarządzaniu kodem związanym z nieprawidłowymi ustawieniami wprowadzonymi przez użytkownika. Kolejną zaletą jest ograniczenie duplikacji kodu. Zarówno funkcjonalność związana z przetwarzaniem danych jak i ta polegająca na przeszukiwaniu mogą wykorzystywać klasy deskryptorów. Dodatkowym atutem podziału kodu na deskryptory jest uproszczenie testów jednostkowych. Testy projektowane będą pod określony deskryptor, dzięki czemu ich kod będzie czytelniejszy i prostszy w utrzymaniu.

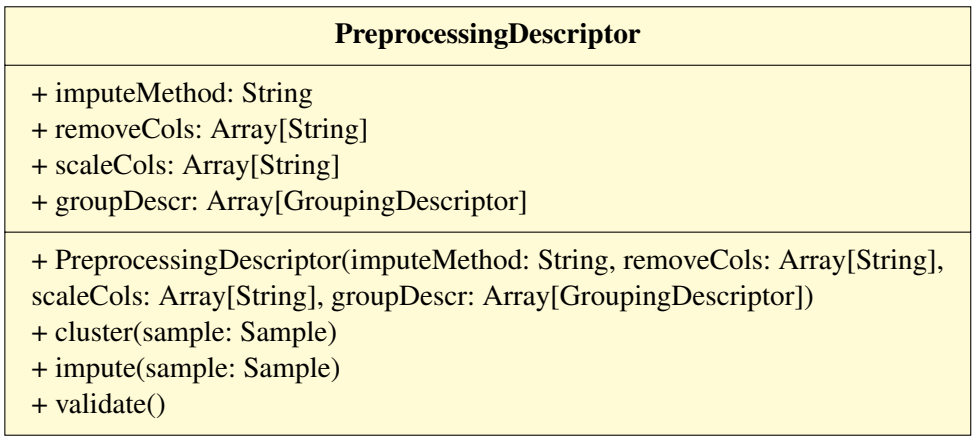
Warto również spojrzeć na kwestię rozproszonego wykonywania obliczeń. Użycie klas, jakimi są deskryptory, umożliwi rozdzielenie kodu odpowiedzialnego za wytwarzanie deskryptorów od kodu przetwarzającego dane zgodnie z opisem zawartym w deskryptorach. Rozdzielenie tych obowiązków posiada bardzo dużą zaletę. Na komputerach wyposażonych w przynajmniej dwa rdzenie procesora aplikacja będzie mogła w prosty sposób przetwarzać dane równolegle. Możliwość ta wynika z faktu, że wygenerowanie pojedynczego deskryptora zajmować będzie znacznie mniej czasu, niż przetworzenie danych związanych z nim. Wątek (lub proces) będzie „rozdawał” opisy operacji innym wątkom (lub procesom) i co jakiś czas odbierał rezultaty obliczeń. Rozwiązanie to pozwoli na skrócenie czasu przeszukiwania.

Klasa PreprocessingDescriptor

W mojej pracy planuję zaimplementować trzy rodzaje deskryptorów. Pierwszym z nich jest klasa *PreprocessingDescriptor* związana ze wstępnym przetwarzaniem danych. Instancje obiektu klasy *PreprocessingDescriptor* posiadają pola zawierające informacje o metodzie radzenia sobie z brakującymi danymi, atrybutach poddawanych skalowaniu itd. Diagram 4.8 przedstawia pola i metody klasy.

Klasa *PreprocessingDescriptor* posiada cztery pola. Pole *imputeMethod* oznacza rodzaj metody imputacji brakujących wartości atrybutów w zbiorze danych. Pole *removeCols* to tablica nazw atrybutów, które powinny zostać usunięte ze zbioru danych w trakcie przebiegu

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00



Rysunek 4.8: Diagram UML klasy *PreprocessingDescriptor*

algorytmu. Pole *scaleCols* jest tablicą nazw atrybutów, które powinny zostać przeskalowane do przedziału [-1, 1]. Ostatnie pole, *groupDescr*, to tablica obiektów klasy *GroupingDescriptor*, opisujących grupowanie atrybutów w zbiorze.

Konstruktor klasy przyjmuje cztery parametry odpowiadające wartościom pól klasy.

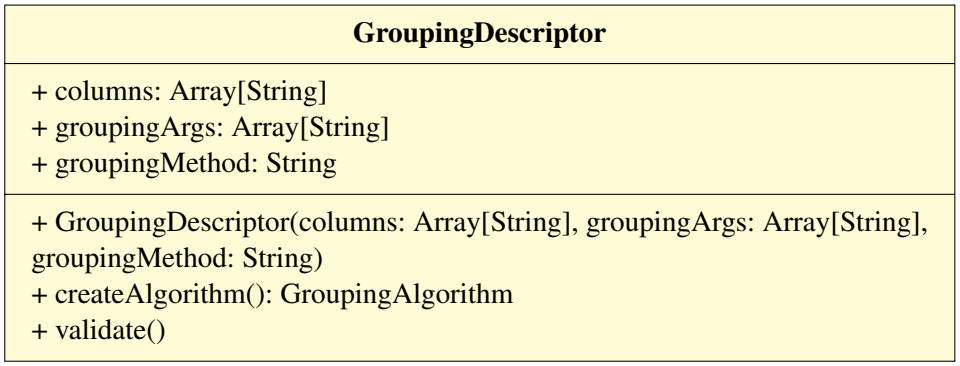
W klasie powinny zostać zaimplementowane dwie metody wytwórcze: *cluster()* oraz *impute()*. Pierwsza z metod przyjmuje jako parametr obiekt klasy *Sample* s_1. Wynikiem metody jest obiekt klasy *Clusterer* z wewnętrznym modelem wytworzonym na obiekcie s_1.

Na podobnej zasadzie działa metoda o nazwie *impute*. Metoda przyjmuje pojedynczy argument - obiekt klasy *Sample* s_2 - i zwraca obiekt klasy *Imputer* nauczony na obiekcie s_2.

Dodatkowo, klasa *PreprocessingDescriptor* posiada metodę *validate()*. Metoda ta ma za zadanie sprawdzić, czy pola klasy mają poprawne wartości. Jeżeli wartości są niepoprawne, metoda rzuca wyjątek typu *DescriptorException*.

Klasa GroupingDescriptor

Następnym deskryptorem jest klasa *GroupingDescriptor*. Przechowuje ona informacje o grupowaniu atrybutów danych z wykorzystaniem pojedynczego, określonego algorytmu grupowania (np. K-Średnich). Poza metodą sprawdzającą, klasa *GroupingDescriptor* oferuje również metodę wytwórczą createAlgorithm. Pola i metody klasy przedstawia diagram 4.9.



Rysunek 4.9: Diagram UML klasy *GroupingDescriptor*

Konstruktor klasy *GroupingDescriptor* posiada trzy parametry. Pierwszy z nich to tablica nazw atrybutów, które mają być poddane grupowaniu. Drugi parametr to tablica wartości, które odpowiadają parametrom wybranego algorytmu grupowania. W zależności od wyboru metody

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

grupowania, tablica ta może mieć różną długość. Ostatnim parametrem konstruktora jest nazwa metody grupowania, np. *k-means*.

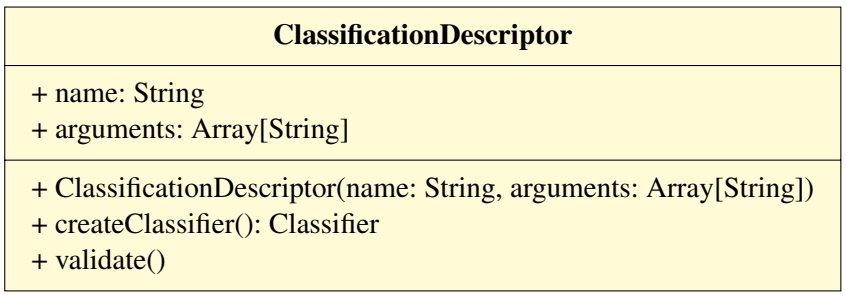
Pola klasy w dokładny sposób odpowiadają parametrom konstruktora klasy.

Klasa posiada metodę wytwórczą *createAlgorithm()*. Zadaniem tej metody jest stworzenie obiektu pozwalającego na grupowanie zgodnie z ustaloną metodą i jej parametrami. Metoda zwraca obiekt implementujący interfejs *GroupingAlgorithm*, który będzie mógł być wykorzystany przez metodę *fit* obiektu klasy *Clusterer* (patrz: 4.2.3).

Ostatnią metodą klasy *GroupingDescriptor* jest *validate()*. Celem tej metody jest upewnienie się, że wszystkie pola klasy mają prawidłowe wartości. W przeciwnym wypadku metoda rzuca wyjątkiem typu *DescriptorException*.

Klasa ClassificationDescriptor

Ostatnim deskryptorem jest klasa, związana z klasyfikacją danych, o nazwie *ClassificationDescriptor*. Klasa ta ma za zadanie przechowywać informacje o algorytmie klasyfikacji użytym podczas przetwarzania. Podobnie jak klasa *PreprocessingDescriptor*, również ta klasa posiada metodę wytwórczą. Różnica polega na tym, że zwraca ona obiekt klasyfikatora implementujący interfejs *Classifier*. Rysunek 4.10 prezentuje pola i metody klasy.



Rysunek 4.10: Diagram UML klasy *ClassificationDescriptor*

Konstruktor klasy przyjmuje dwa parametry. Pierwszym z nich jest nazwa metody klasyfikacji, np. *tree* lub *randomforest*. Drugim parametrem konstruktora jest tablica wartości będąca parametrami wybranej metody klasyfikacji. W zależności od wyboru metody, długość tablicy może być różna.

Klasa posiada dwa pola - *name* oraz *arguments* - które odpowiadają parametrom konstruktora klasy.

Zadaniem metody wytwórczej *createClassifier()* jest zwrócenie obiektu implementującego interfejs *Classifier* i reprezentującego algorytm klasyfikacji opisywany przez deskryptor.

Metoda *validate()* klasy *ClassificationDescriptor* sprawdza poprawność pól klasy. Jeżeli wartości pól są nieprawidłowe, metoda rzuca wyjątkiem *DescriptorException*.

4.2.5 Generowanie algorytmów wewnątrz rodzin

Kluczową cechą systemu jest rozwijanie opisu rodziny algorytmów w zbiór algorytmów, które mają być poddane ocenie. Zgodnie z założeniami opisanymi w rozdziale poświęconym wymaganiom systemu, aplikacja umożliwiać będzie sprecyzowanie rodziny algorytmów, a następnie przystąpienie oceniać algorytmy wchodzące w skład rodziny. Aplikacja sprawdzać będzie poszczególne kombinacje operacji związanych z przetwarzaniem wstępnym oraz klasyfikacją. Przeszukiwanie sprawdzi wszystkie warianty, które będą odpowiadać opisowi wprowadzonemu przez użytkownika. W zależności od opisu, czas wykonywania operacji przewiduję na od kilku sekund do kilkunastu godzin. Z tego względu postarałem się zaprojektować przeszukiwanie w sposób umożliwiający łatwe sprawdzenie jego poprawności. Dodatkowo, przygotowany

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

przeze mnie projekt rozdziela przeszukiwanie na luźno powiązane fragmenty, uproszczając tym samym bardzo skomplikowaną implementację.

Rozwijanie rodzin, najprościej mówiąc, polegać będzie na generowaniu wszystkich kombinacji metod przetwarzania wstępnego i klasyfikacji, zgodnie z opisem zawartym w deskryptorach (patrz: 4.2.4).

Pomysł ten postanowiłem sprowadzić do sekwencji zagnieżdżonych pętli (rysunek 2). Każda pętla odpowiadać będzie osobnej operacji związanej z przetwarzaniem danych. Zadaniem wszystkich pętli będzie przyrostowe tworzenie opisu czynności do wykonania. Polegać to będzie na uzupełnianiu pól deskryptorów informacjami związanymi z etapami przetwarzania odpowiadającym danym pętlom. Po uzupełnieniu odpowiedniego pola deskryptora, każda pętla przekaże go do pętli wewnętrznej. Dzięki temu, podczas każdej iteracji pętli najgłębiej zagnieżdżonej otrzymamy deskryptory opisujące konkretną kombinację elementów przetwarzania. Informacja zawarta w deskryptorach będzie wystarczająca aby wykonać pojedynczy algorytm i otrzymać wynik walidacji krzyżowej.

```
imputeSpace ← kombinacje metod imputacji;
removeSpace ← kombinacje usuwania atrybutów;
scaleSpace ← kombinacje atrybutów do przeskalowania;
groupingSpace ← kombinacje użycia wybranych metod grupowania;
classificationSpace ← kombinacje metod klasyfikacji;
A ← ∅;
p0 ← PreprocessingDescriptor;
Dla każdego p1 ∈ imputeSpace.generate(p0)
    Dla każdego p2 ∈ removeSpace.generate(p1)
        Dla każdego p3 ∈ scaleSpace.generate(p2)
            Dla każdego p4 ∈ groupingSpace.generate(p3)
                Dla każdego c ∈ classificationSpace.generate()
                    A ← A ∪ {(p4, c)};
                Koniec
            Koniec
        Koniec
    Koniec
Koniec
Zwróć A
```

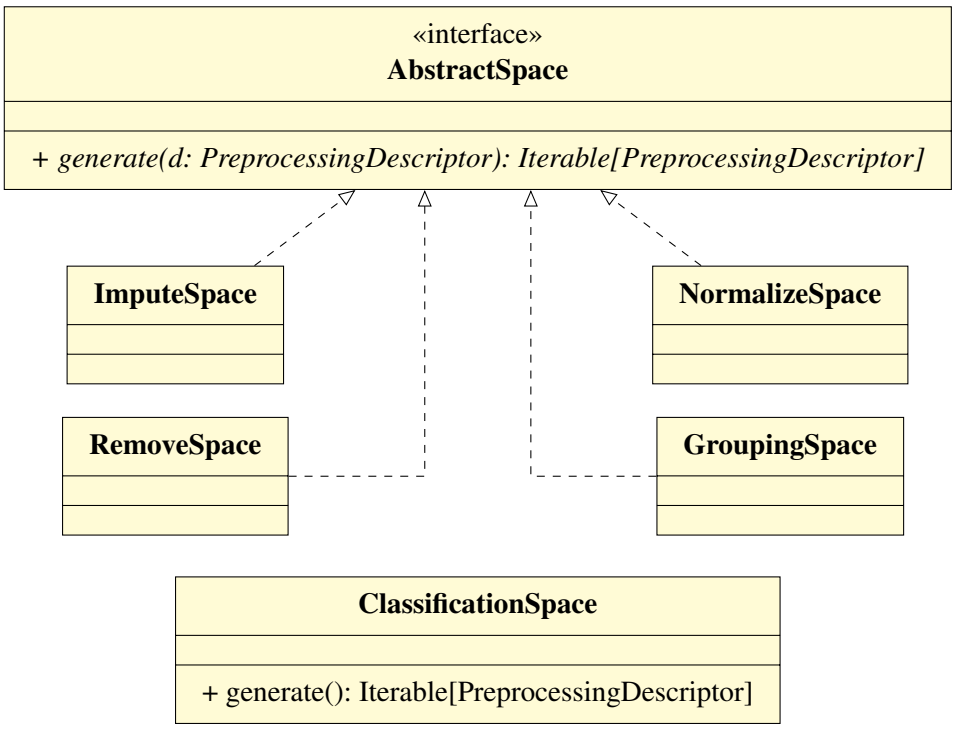
Algorithm 2: Rozwijanie opisu rodzin algorytmów w zbiór konkretnych algorytmów

Należy wspomnieć, że kolejność zagnieżdżania pętli odgrywa tu rolę. Dzieje się tak, ponieważ niektóre operacje mogą mieć wpływ na decyzje o wykonaniu innych operacji. Przykładowo, po usunięciu atrybutu o nazwie *VEGF* nie jest możliwe przeskalowanie atrybutu o tej samej nazwie (atrybutu już nie ma w zbiorze danych) - z tego powodu pętla odpowiadająca skalowaniu musi być umieszczona wewnątrz pętli odpowiadającej usuwaniu atrybutów.

W celu uproszczenia kodu związanego z obsługą każdej z pętli, iterowanie po każdym poziomie zagnieżdżenia realizowane jest przy pomocy obiektów specjalnych klas. Każda z klas wykorzystuje wzorzec projektowy *Iterator* [7]. Klasy wytwarzające kolekcje obiektów typu *PreprocessingDescriptor* implementują interfejs *AbstractSpace*. Diagram klas 4.11 ilustruje zależności między klasami.

Interfejs *AbstractSpace* udostępniać będzie tylko jedną metodę o nazwie *generate*. Argumentem metody będzie deskryptor - w założeniu pochodzący z pętli z wyższego poziomu

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00



Rysunek 4.11: Diagram UML klas służących do generowania deskryptorów na podstawie opisu rodziny algorytmów

zagnieżdżenia. Metoda zwracać będzie ten sam deskryptor lecz uzupełniony o informacje związane z danym etapem przetwarzania.

Aplikacja nadzorująca wykonywanie obliczeń generuje kolejne pary deskryptorów - przetwarzania wstępnego i klasyfikacji - i rozdziela je pomiędzy procesy zajmujące się obliczeniami. Dokładny opis tego procesu znajduje się w części 4.4.

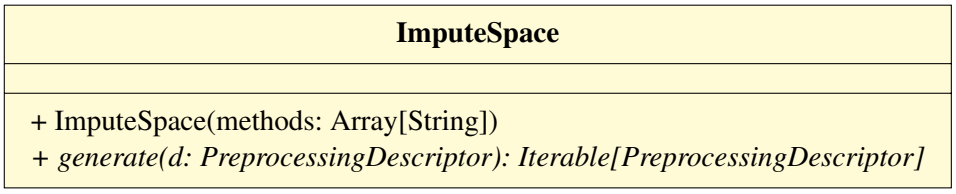
Klasa ImputeSpace

Zadaniem klasy *ImputeSpace* jest generowanie deskryptorów przetwarzania wstępnego z ustaloną metodą imputacji brakujących wartości atrybutów.

Konstruktor klasy przyjmuje jeden parametr będący tablicą nazw metod imputacji. Dozwolone nazwy metod to mean i median.

Klasa posiada tylko jedną metodę o nazwie *generate*. Metoda ta ma pojedynczy argument - deskryptor przetwarzania wstępnego d. Metoda zwraca kolekcję obiektów typu *PreprocessingDescriptor* z ustalonymi metodami imputacji danych.

Diagram 4.12 przedstawia klasę *ImputeSpace*.

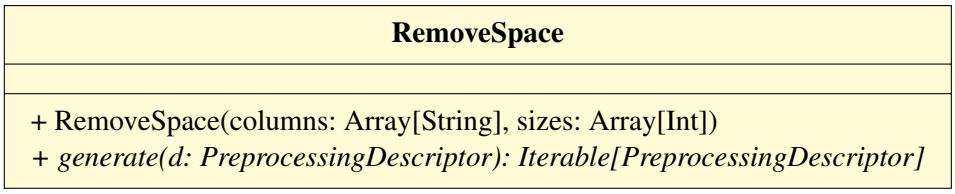


Rysunek 4.12: Diagram UML klasy *ImputeSpace*

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Klasa RemoveSpace

Klasa RemoveSpace upraszcza generowanie wszystkich kombinacji atrybutów do usunięcia, zgodnie z parametrami wybranymi przez użytkownka. Diagram klasy jest zilustrowany na rysunku 4.13.



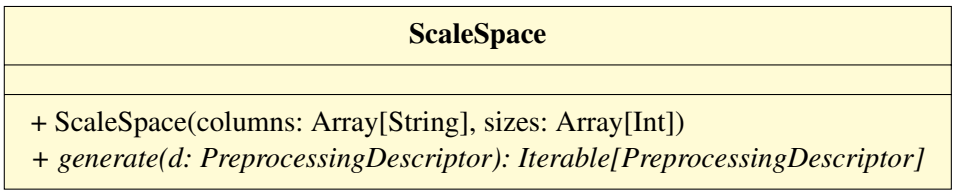
Rysunek 4.13: Diagram UML klasy *RemoveSpace*

Konstruktor klasy przyjmuje dwa parametry. Pierwszym z nich jest zmienna *columns* będąca tablicą nazw atrybutów, spośród których mają być wygenerowane kombinacje. Drugi argument, *sizes*, to tablica rozmiarów podzbiorów, które obiekt klasy *RemoveSpace* ma wytworzyć.

Za generowanie kombinacji atrybutów do usunięcia odpowiedzialna jest metoda *generate()*. Przyjmuje ona pojedynczy deskryptor, którego zmodyfikowane kopie zwraca jako wynik. Metoda ta, dla każdej liczności podzbioru l zawartej w tablicy *sizes*, zwróci obiekty klasy *PreprocessingDescriptor* zawierające informacje o usunięciu wszystkich podzbiorów atrybutów *columns* o liczności l. Przykładowo, dla listy atrybutów $\{a, b, c, d\}$ oraz listy krotności $\{1, 2\}$ algorytm wytworzy $\binom{4}{1} + \binom{4}{2} = 10$ kombinacji.

Klasa ScaleSpace

Klasa *ScaleSpace* przypomina w działaniu klasę *RemoveSpace*. Zadaniem klasy jest wytwarzanie deskryptorów przetwarzania wstępnego posiadających informację o przeskalowaniu wybranej rodziny atrybutów. Diagram klasy pokazano na rysunku 4.14.



Rysunek 4.14: Diagram UML klasy *ScaleSpace*

Konstruktor klasy działa analogicznie do konstruktora klasy *RemoveSpace*.

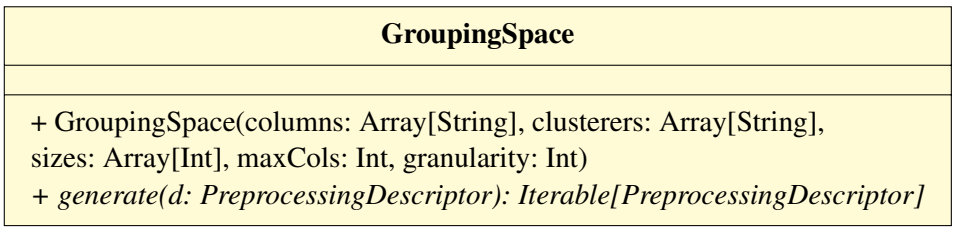
Metoda *generate()* przyjmuje jako parametr deskryptor przetwarzania d. Zwraca ona kolekcję obiektów klasy *PreprocessingDescriptor*, z których każdy ma ustaloną informację o atrybutach, które mają być przeskalowane. Metoda *generate()* uwzględnia ewentualny brak niektórych atrybutów spowodowany ich „usunięciem”, opisanym w deskryptorze d.

Klasa GroupingSpace

Klasa *GroupingSpace* upraszcza generowanie deskryptorów przetwarzania zawierających informacje o grupowaniu atrybutów zgodnie z ustawieniami użytkownika. Szczegółowy wygląd klasy prezentuje diagram 4.15.

Konstruktor klasy przyjmuje parametry odpowiadające opisowi fazy grupowania ustalonej przez użytkownika. Pierwszym parametrem jest tablica nazw atrybutów, wśród których ma

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00



Rysunek 4.15: Diagram UML klasy *GroupingSpace*

nastąpić grupowanie. Drugi parametr to tablica nazw metod grupowania, które powinny być użyte (np. *k-means*). Następny parametr to tablica liczb $S = s_1, s_2, \ldots, s_n$, gdzie s_i oznacza liczbę grupowań przeprowadzanych jednocześnie. Pozwala to na grupowanie kilku rozłącznych podzbiorów atrybutów na raz. Czwarty parametr określa, ile maksymalnie atrybutów może być połączonych przez algorytm grupowania. Ostatni parametr to *granularność* doboru parametrów poszczególnych metod grupowania (patrz: 3.1.1).

Metoda *generate()* zwraca kolekcję deskryptorów przetwarzania z wypełnionymi polami opisującymi fazę grupowania w każdym z nich.

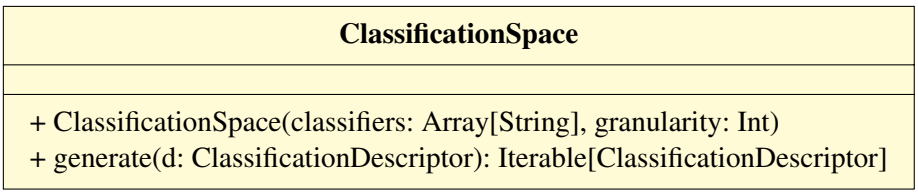
Klasa ClassificationSpace

Ostatnią klasą pomocną przy rozwijaniu opisów rodzin algorytmów jest klasa *ClassificationSpace*.

Konstruktor klasy przyjmuje dwa parametry. Pierwszy parametr to tablica nazw metod klasyfikacji, które mają być użyte wewnątrz gruy algorytmów (każdy pojedynczy algorytm używa tylko jednej metody klasyfikacji). Drug parametr to *granularność* doboru parametrów klasyfikacji.

Metoda *generate()* zwraca kolekcję obiektów klasy *ClassificationDescriptor*. W odróżnieniu od metod zaimplementowanych w klasach używających interfejsu *AbstractSpace*, metoda ta nie przyjmuje żadnych parametrów.

Klasa jest przedstawiona na diagramie 4.16.



Rysunek 4.16: Diagram UML klasy *ClassificationSpace*

Klasa SearchSpace

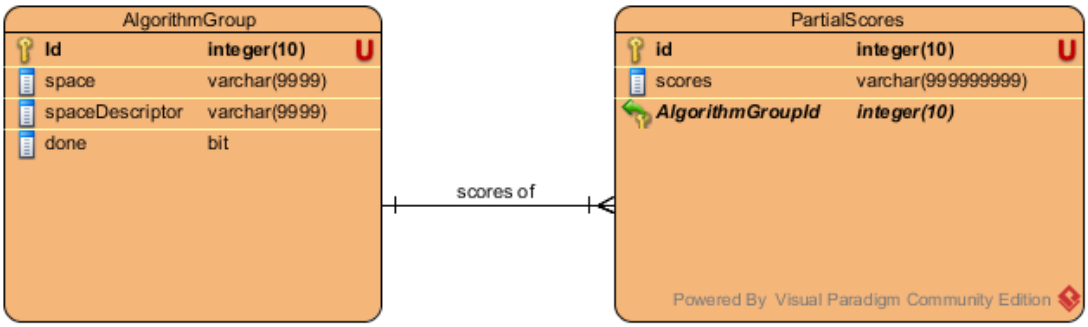
Iterowanie z wykorzystaniem powyższych klas jest zaimplementowane w klasie *SearchSpace*. Klasa ta, będąca kolejnym przykładem iteratora, pozwoli na łatwe generowanie kolejnych deskryptorów. Konstruktor klasy przyjmować będzie obiekty klas *ImputeSpace*, *RemoveSpace*, *ScaleSpace*, *GroupingSpace* oraz *ClassificationSpace*. Przy każdej iteracji obiekt klasy *SearchSpace* zwróci parę deskryptorów – obiekty klas *PreprocessingDescriptor* oraz *ClassificationDescriptor*.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

4.3 Struktura przechowywanych danych

Projektując system informatyczny, bardzo ważne jest zdefiniowanie struktury danych, które system będzie przechowywał. Wczesne spojrzenie na dane i ich rodzaj pozwala na dokładną analizę tych fragmentów aplikacji, które z tych danych będą korzystać.

W związku z tym, że system używa relacyjne bazy danych PostgreSQL, schemat danych jest ściśle ustalony. Diagram modelu danych pokazuje rysunek 4.17.



Rysunek 4.17: Model danych dla bazy relacyjnej

Tabela *AlgorithmGroup* stanowi reprezentację obliczeń związanych z pojedynczą rodziną algorytmów. Atrybut *space* jest zserializowanym obiektem klasy *SearchSpace*. Atrybut ten jest potrzebny, aby program przetwarzający informacje o rodzinach miał dostęp do szczegółów każdej z nich. Atrybut *spaceDescriptor* to czytelny opis rodziny algorytmów. Ostatni atrybut o nazwie *done* przechowuje wartość 0 lub 1 i wskazuje czy obliczenia dla danej rodziny zostały ukończone, czy nie. Kolumna *id* to unikalny identyfikator wiersza.

Dla każdego wiersza w tabeli AlgorithmGroup istnieje przynajmniej jeden wiersz w tabeli PartialScores. Tabela ta przechowuje cząstkowe wyniki obliczeń pochodzące z pojedynczego programu wykonującego obliczenia. Zbiór wszystkich wierszy tabeli PartialScores P_a odpowiadający konkretnemu wierszowi tabeli AlgorithmGroup a zawiera wszystkie wyniki oceny klasyfikacji rodziny algorytmów opisanej w a.

Kolumna *id* tabeli *PartialScores* to unikalny identyfikator wiersza. Pole *scores* to zserializowana tablica trójek wartości - miary wrażliwości, precyzji i F1. *AlgorithmGroupId* to klucz obcy tabeli.

4.4 Program nadzorujący obliczenia

Program nadzorujący obliczenia ma za zadanie pobrać od użytkownika opis rodziny algorytmów do oceny, dokonać oceny oraz zapisać wyniki do bazy danych. Część ta skupia się na opisie algorytmów oraz klas użytych w programie nadzorującym.

4.4.1 Interfejs użytkownika

Nadzorca jest aplikacją konsolową. Szczegóły dotyczące uruchomienia obliczeń przekazywane są jako argumenty linii poleceń. Tabela 4.1 zwiera zwięzły opis argumentów programu.

Jak można zauważyć, argumenty programu umożliwiają sprecyzowanie rodziny algorytmów tak, jak jest to opisane w rozdziale poświęconym założeniom i wymaganiom systemu. Interfejs pozwala tworzyć nowe ziarna pseudolosowe używane do podziału zbioru danych podczas walidacji krzyżowej. Nadzorca zapisuje informacje o wynikach oceny w bazie danych i, w razie ponownego uruchomienia dla tej samej rodziny algorytmów, nie wykonuje żadnej akcji. Argument *-r* pozwala „zresetować” stan wiedzy o wynikach i uruchomić obliczenia jeszcze raz.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Parametr	Opis
-f *metody*	Lista metod imputacji danych.
-rc *atrybuty*	Lista atrybutów przeznaczonych do usunięcia.
-rs *rozmiary*	Lista rozmiarów podzbiorów atrybutów przeznaczonych do usunięcia.
-nc *atrybuty*	Lista atrybutów przeznaczonych do przeskalowania.
-ns *rozmiary*	Lista rozmiarów podzbiorów atrybutów przeznaczonych do przeskalowania.
-qc *atrybuty*	Lista atrybutów, które mogą być użyte podczas grupowania.
-qa *algorytmy*	Lista algorytmów grupowania, których kombinacje mają być użyte.
-qs *liczności*	Lista liczności algorytmów grupowania, które mają być uruchomione jednocześnie na rozłącznych podzbiorach atrybutów.
-qm *liczba*	Maksymalna liczba atrybutów grupowanych jednocześnie przez pojedynczy algorytm grupowania.
-qg *granularność*	Granularność algorytmów grupowania.
-cg *granularność*	Granularność algorytmów klasyfikacji.
-gs *rozmiar*	Maksymalny rozmiar pojedynczego bloku algorytmów.
-ws *rozmiar*	Maksymalny rozmiar puli zadań obserwowanych przez nadzorcę.
-r	Nadpisz istniejące wyniki dla rodziny algorytmów.
-s	Utwórz nowe ziarno pseudolosowe.

Tablica 4.1: Argumenty programu nadzorującego obliczenia.

Argumenty programu parsowane są przez klasę o nazwie *Parser*. Zadaniem klasy jest przyjęcie ciągów znaków (argumentów), przeanalizowanie go i wyprodukowanie struktury danych typu *CmdArguments* zawierającej pola odpowiadające poszczególnym argumentom.

4.4.2 Zarządzenie obliczeniami równoległymi

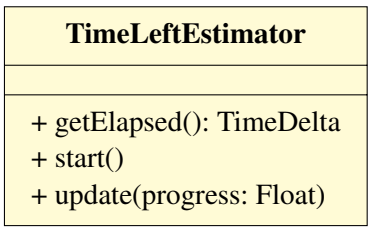
Po otrzymaniu parametrów zgromadzonych w obiekcie klasy *CmdArguments*, aplikacja może rozpocząć właściwe działanie. W głównym wątku aplikacji obiekt app klasy *CalculateApplication* odczytuje z bazy danych informacje o tym czy obliczenia powinny być rozpoczęte, czy wznowione od pewnego momentu. Następnie tworzony jest obiekt klasy *TimeLeftEstimator* służący do estymacji czasu trwania reszty obliczeń. Kolejnym korkiem jest utworzenie obiektu $procedure$ klasy *EvaluationProcedure*, który zawiera w sobie całą logikę zarządzania obliczeniami rozproszonymi. Obiekt ten powołuje do życia nowy wątek w którym wysyła do programów-robotników zlecenia oceny algorytmów. Obiekt app na bieżąco monitoruje postęp działania obiektu $procedure$ i wyświetla go na ekranie razem z estymatą zakończenia obliczeń. Ilekroć obiekt $procedure$ odbierze oceny pewnych algorytmów, wywołuje funkcję „callback" obiektu app, która zapisuje oceny w bazie danych. Program kończy swe działanie gdy obliczenia się zakończą lub gdy użytkownik wciśnie kombinację klawiszy *Ctrl+C*.

Klasa TimeLeftEstimator

Klasa *TimeLeftEstimator* służy do przewidzenia, kiedy obliczenia się zakończą, na podstawie tempa postęp tych obliczeń.

Metoda *start()* rozpoczyna pomiar czasu przez estymator. Metoda *update()* przyjmuje jeden parametr - postęp obliczeń wyrażony jako liczba z przedziału [0, 1]. Metoda ta uaktualnia

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00



Rysunek 4.18: Diagram UML klasy *TimeLeftEstimator*

wewnętrzny stan obiektu. Metoda o nazwie *getElapsed* zwraca domniemany czas potrzebny na zakończenie obliczeń.

Klasa CalculateApplication

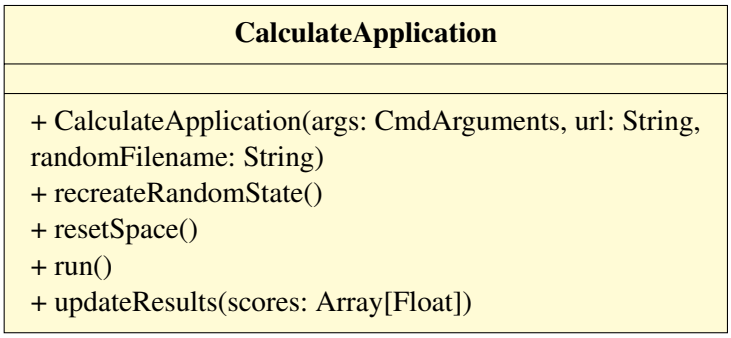
Podstawę prezentacji i komunikacji z użytkownikiem stanowi klasa *CalculateApplication*. Konstruktor klasy pobiera trzy parametry. Pierwszym z nich jest obiekt klasy *CmdArguments* reprezentujący argumenty linii poleceń. Następnym parametrem jest URL serwera bazy danych. Ostatnim parametrem jest ścieżka do pliku z zapisanym ziarnem pseudolosowym.

Metoda *resetSpace()* usuwa z bazy danych wpis o obliczeniach dotyczących aktualnej rodziny algorytmów wybranej przez użytkownika.

Ziarno pseudolosowe może być wygenerowane ponownie i zapisane do pliku dzięki metodzie *recreateRandomState()*.

Ilekroć nadzorca odbiera nowe wyniki obliczeń, następuje potrzeba zapisania ich do bazy danych. Metoda *updateResults()* jest funkcją typu callback obiektu klasy *CalculateApplication* i jest wywoływana z osobnego wątku przez obiekt klasy *EvaluationProcedure* ilekroć pojawią się nowe oceny algorytmów.

Metoda *run()* odpowiada za uruchomienie obliczeń w programie. Całość prezentuje rysunek 4.19.



Rysunek 4.19: Diagram UML klasy *CalculateApplication*

Klasa EvaluationProcedure

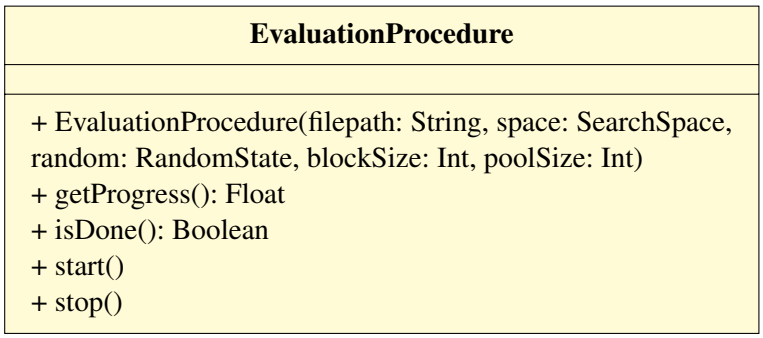
Zarządzanie przeprowadzaniem obliczeń powinno przebiegać w następujący sposób. Nadzorca generuje wszystkie algorytmy wchodzące w skład zdefiniowanej przez użytkownika rodziny algorytmów. Następnie tworzona jest pusta pula zadań, które są aktualnie wykonywanie przez robotników. Nadzorca przydziela algorytmy do zadań, które następnie umieszcza w puli. Dodanie zadania do puli jest jednoznaczne z wysłaniem zadania do systemu kolejkowego i rozpoczęciem obserwowania jego postępów. Pula zadań ma ustalony maksymalny rozmiar, np. 100 zadań monitorowanych jednocześnie. Gdy któreś zadanie zostanie ukończone, nadzorca odbiera

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

wyniki z systemu kolejkowego, umieszcza je w bazie danych i dodaje do puli nowe zadanie. Nadzorca kończy pracę gdy zadania się wyczerpią i gdy wszystkie zadania zostaną ukończone.

Jak zostało to powiedziane w części 3.1.2, prowadząc obliczenia na wielu maszynach, należy mieć na uwadze wpływ opóźnień w sieci na wydajność systemu. Proces nadzorujący obliczenia powinien zatem wysyłać bloki deskryptorów, a nie pojedyncze deskryptory. Przesyłanie informacji o algorytmach do oceny w dużych zbiorach minimalizuje narzut związany z komunikacją sieciową.

Klasa *EvaluationProcedure* jest odpowiedzialna za zarządzanie wykonywaniem obliczeń w nadzorcy. Konstruktor klasy przyjmuje pięć parametrów. Pierwszym z nich jest ścieżka do pliku z danymi. Drugi parametr to to obiekt klasy *SearchSpace*. Kolejny parametr to obiekt reprezentujący ziarno pseudolosowe. Czwartym parametrem konstruktora jest rozmiar bloku wysyłanych deskryptorów. Ostatnim parametrem jest rozmiar puli zadań.

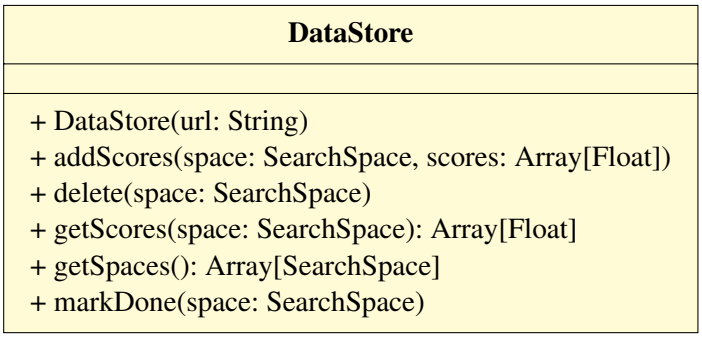


Rysunek 4.20: Diagram UML klasy *EvaluationProcedure*

Zarządzanie obliczeniami rozpoczyna się po wywołaniu metody *start()*. Metoda *stop()* wstrzymuje pracę. Aby uzyskać informacje o całkowitym postępie obliczeń należy użyć metody *getProgress()*. Zwraca ona liczbę z przedziału [0, 1] odpowiadającą zaawansowaniu wykonywanych obliczeń. Metoda *isDone()* pozwala stwierdzić czy obliczenia zostały ukończone.

4.4.3 Warstwa dostępu do danych

Dostęp do danych w bazie zapewnia obiekt klasy DataStore. Metody klasy umożliwiają dodawanie informacji o analizowanej rodzinie algorytmów, dodawanie wyników cząstkowych do badanej rodziny, usuwanie rodziny oraz przeglądanie rodzin algorytmów zapisanych w bazie. Klasę DataStore przedstawia rysunek 4.21.



Rysunek 4.21: Diagram UML klasy *DataStore*

Konstruktor klasy *DataStore* przyjmuje jeden argument - adres URL serwera bazy danych.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Dodawanie wyników do bazy realizowane jest poprzez wołania metody *addScores()*. Pierwszym parametrem metody jest obiekt klasy *SearchSpace* opisujący rodzinę algorytmów, do której należy dodać wyniki. Drugim parametrem metody jest tablica wartości ocen jakości klasyfikacji. Metoda *addScores()* dodaje wyniki do istniejącego wiersza w tabeli *AlgorithmGroup*. Jeżeli wiersz w tabeli nie istnieje, zostanie on utworzony przez metodę.

Zadaniem metody o nazwie delete jest usunięcie z bazy informacji o obliczeniach dotyczących wybranej rodziny algorytmów. Parametrem metody jest obiekt klasy *SearchSpace* reprezentujący tę rodzinę.

Metoda *getScores()* zwraca tablicę wyników ocen wybranej rodziny algorytmów. Parametrem metody jest obiekt klasy *SearchSpace*.

W celu uzyskania informacji o wszystkich rodzinach algorytmów, dla których obliczenia były prowadzone, należy wywołać metodę *getSpaces()*. Zwróci ona tablicę obiektów klasy *SearchSpace*.

Ostatnią metodą publiczną klasy *DataStore* jest *markDone()*. Metoda to oznacza rodzinę algorytmów jako taką, dla której wszystkie obliczenia zostały ukończone.

4.5 Program liczący

Zadaniem programu liczącego jest odbieranie zadań z systemu kolejkowego, wykonywanie niezbędnych obliczeń i zwracanie wyników do systemu kolejkowego.

Odbieranie zadań jest zapewnione przez biblioteki programistyczne dostarczone przez twórców systemu kolejkowego. Wykonywanie obliczeń polega na wytworzeniu zbioru metryk klasyfikacji dla każdego algorytmu zawartego w bloku dostarczonym razem z zadaniem. Metryki są łączone w jeden zbiór i wysyłane z powrotem do systemu kolejkowego jako rezultat pojedynczego zadania.

Program liczący używa biblioteki *mltool* w celu przeprowadzenia obliczeń oraz bibliotek związanych z systemem kolejkowym aby odebrać zadanie i wysłać wyniki.

4.6 Prezentowanie wyników obliczeń

Celem obliczania metryk jakości klasyfikacji dla rodzin algorytmów jest ich późniejsza analiza. Badanie polega na przyglądaniu się cechom wyników wewnątrz rodzin (np. wartość średnia, odchylenie standardowe, kształt rozkładu) oraz na porównaniu wyników między rodzinami. Do realizacji tych zadań służą programy *browse* i *compare*.

4.6.1 Program browse

Program *browse* służy do przeglądu otrzymanych wyników jakości klasyfikacji rodzin algorytmów. Program umożliwia przegląd wyników wybranych rodzin algorytmów lub wszystkich z nich.

Interfejs użytkownika

Użytkownik ma wpływ na działanie programu poprzez konsolowy interfejs. Argumenty programu *browse* prezentuje tabela 4.2.

Wyświetlanie informacji o wynikach rodzin działa na dwa sposoby. Tryb pierwszy wyświetla szczegółową listę rodzin algorytmów oraz informacje o wynikach tj. średnie wartości metryk, liczba algorytmów w rodzinie. Tryb drugi wyświetla histogramy wybranej metryki dla ustalonych rodzin algorytmów. Do przełączania trybów pracy służy argument *-a*.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Parametr	Opis
-a *akcja*	Wybór metody przeglądania. Możliwe wartości akcji to *list* i *plot*.
-ids *identyfikatory*	Lista identyfikatorów rodzin algorytmów, o których informacje należy wyświetlić.
-m *metryka*	Rodzaj metryki, którą należy wyświetlić.

Tablica 4.2: Argumenty programu *browse*.

Program *browse* pozwala wybrać rodziny algorytmów, które mają być pokazane na ekranie. Do wyboru służy opcja *-ids*. Jeżeli opcja ta nie jest sprecyzowana, zostaną wyświetlone wszystkie rodziny.

Podczas wyświetlania histogramów wybranych metryk należy wybrać rodzaj metryki. Służy do tego argument *-m*. Jeżeli argument ten nie jest jawnie zdefiniowany, domyślnym rodzajem oceny jest metryka F1.

Struktura programu

Program *browse* posiada klasę o nazwie *BrowseCmdParser* odpowiedzialną za parsowanie argumentów linii poleceń. Metoda *parse()* klasy *BrowseCmdParser* zwraca strukturę zawierającą pola odpowiadające argumentom linii poleceń.

Aplikacja używa biblioteki *mltool* w celu przetwarzania wyników poszczególnych rodzin (np. liczenie histogramu) oraz bibliotek umożliwiających komunikację z bazą danych.

4.6.2 Program compare

Porównywanie wyników rodzin algorytmów między sobą jest realizowane przez program o nazwie *compare*. Efektem działania programu jest ranking rodzin zgodny z założeniami opisanymi w części 3.1.2.

Interfejs programu

Program *compare* jest aplikacją konsolową. Argumenty linii poleceń programu pokazane są w tabeli 4.3.

Parametr	Opis
-p *poziom*	Poziom istotności (od 0 do 1) dla testu Manna-Whitneya-Wilcoxona.
-o *metryka*	Metryka wg której tworzony jest ranking.
-c *liczba*	Liczba najlepszych pozycji w rankingu do wyświetlenia.

Tablica 4.3: Argumenty programu *compare*.

Struktura aplikacji

Klasa *CompareCmdParser* pozwala na parsowanie argumentów linii poleceń i otrzymanie struktury zawierającej odpowiadające im pola.

Program *compare* używa biblioteki *mltool*, bibliotek umożliwiających komunikację z bazą danych oraz bibliotek matematycznych posiadających implementację testu Manna-Whitneya-Wilcoxona.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

4.7 Testy

Biblioteka obliczeniowa *mltool* stanowi fundament całej aplikacji. Wyniki działania biblioteki decydować będą o rezultatach badań, jakie przeprowadzać będzie użytkownik. Z tego powodu niezmiernie istotne jest upewnienie się, że biblioteka obliczeniowa działa poprawnie.

W celu sprawdzenia tego istotnego warunku, poprawność działania klas w bibliotece będzie sprawdzana przy pomocy testów jednostkowych. Dzięki modularnej strukturze biblioteki opisanej w punkcie 4.2.1, możliwe jest oddzielne testowanie poszczególnych klas biblioteki. Weryfikacja poprawności działania poszczególnych fragmentów biblioteki umożliwi wyeliminowanie większości możliwych błędów.

Tabela 4.4 prezentuje opis testów jednostkowych badających poprawność działania części biblioteki odpowiedzialnej za odczyt i przetwarzanie wstępne danych. Testy klas-modeli pokazuje tabela 4.5. W tabeli 4.6 zaprezentowane są testy badające poprawność deskryptorów. Testy klas generujących algorytmy wewnątrz rodzin algorytmów opisane są w tabeli 4.7.

Nazwa klasy	Opis testów
XlsFile	Badanie poprawności odczytu danych prawidłowych oraz badanie reakcji na próbę odczytu danych nieprawidłowych.
Sample	Badanie poprawności odczytu danych prawidłowych oraz badanie reakcji na próbę odczytu danych nieprawidłowych. Sprawdzenie poprawnego działania metod *getNormalizer*, *removeColumns*, *impute*, *normalize*, *mergeColumns*.

Tablica 4.4: Opis testów klas biblioteki *mltool* związanych z odczytem i reprezentacją danych.

Nazwa klasy	Opis testów
Imputer	Sprawdzenie poprawnego działania metod *fit* i *transform*.
Normalizer	Sprawdzenie poprawnego działania metod *fit* i *transform*.
Clusterer	Sprawdzenie poprawnego działania metod *fit* i *transform*.
NaiveBayesClassifier	Badanie poprawnego działania metody *fit*. Sprawdzenie czy predykcja metodą *predict()* spełnia podstawowe założenia dotyczące przewidywania kategorii.
DecisionTreeClassifier	Sprawdzenie poprawnego działania metody *fit*. Sprawdzenie czy predykcja metodą *predict()* spełnia podstawowe założenia dotyczące przewidywania kategorii.
SVMClassifier	Badanie poprawnego działania metody *fit*. Sprawdzenie czy predykcja metodą *predict()* spełnia podstawowe założenia dotyczące przewidywania kategorii.
RandomForestClassifier	Sprawdzenie poprawnego działania metody *fit*. Sprawdzenie czy predykcja metodą *predict()* spełnia podstawowe założenia dotyczące przewidywania kategorii.

Tablica 4.5: Opis testów klas-modeli biblioteki *mltool*.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Nazwa klasy	Opis testów
PreprocessingDescriptor	Wywoływanie metody *validate* dla poprawnych i niepoprawnych ustawień deskryptora. Sprawdzanie czy dla danych nieprawidłowych metoda *validate* rzuca wyjątkiem DescriptorException. Badanie czy metody *cluster* i *impute* zwracają właściwe wyniki.
GroupingDescriptor	Uruchamianie metody *validate* dla poprawnych i niepoprawnych ustawień deskryptora. Sprawdzanie czy dla danych nieprawidłowych metoda *validate* rzuca wyjątkiem DescriptorException. Sprawdzanie czy metoda *createAlgorithm* zwraca poprawny wynik.
ClassificationDescriptor	Sprawdzenie czy metoda *createClassifier* działa poprawnie. Wywoływanie metody *validate* dla poprawnych i niepoprawnych ustawień deskryptora. Sprawdzanie czy dla danych nieprawidłowych metoda *validate* rzuca wyjątkiem DescriptorException.

Tablica 4.6: Opis testów deskryptorów w bibliotece *mltool*.

Nazwa klasy	Opis testów
ImputeSpace	Sprawdzenie czy metoda *generate* wytwarza wyniki zgodne z parametrami obiektu.
RemoveSpace	Wywołanie metody *generate* dla obiektu klasy *RemoveSpace* i sprawdzenie czy zwrócone obiekty klasy *PreprocessingDescriptor* są zgodne z założeniami.
ScaleSpace	Badanie czy efektem wywołania metody *RemoveSpace* jest tablica prawidłowych obiektów klasy *PreprocessingDescriptor*. Sprawdzenie tego warunku dla wariantu z usuwaniem atrybutów i bez usuwania.
GroupingSpace	Sprawdzenie czy metoda *generate* zwraca kolekcję obiektów zgodną z ustawieniami obiektu *GroupingSpace*.
ClassificationSpace	Wywołanie metody *generate* i zbadanie czy wygenerowanie deskryptory klasyfikacji są prawidłowe.
SearchSpace	Sprawdzenie czy metoda *generate* generuje wszystkie warianty par deskryptorów przetwarzania wstępnego i klasyfikacji.

Tablica 4.7: Opis testów klas upraszczających generowanie algorytmów w bibliotece *mltool*.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

5. Elementy implementacji systemu

Rozdział ten skupia się na wybranych fragmentach implementacji systemu, które uznałem za szczególnie ciekawe.

5.1 Użyte technologie

Część ta jest poświęcona technologii, w której został zaimplementowany system komputerowy będący jednym z celów pracy magisterskiej.

5.1.1 Język programowania Python

System został w całości zaimplementowany z wykorzystaniem języka programowania *Python*. *Python* jest interpretowanym językiem programowania wysokiego poziomu, o rozbudowanym pakiecie bibliotek standardowych, którego główną ideą jest czytelność i klarowność kodu źródłowego. Jego składnia jest przejrzysta i zwięzła. Python jest dynamicznie typowany i posiada automatyczne zarządzanie pamięcią. Język ten jest zaprojektowany pod kątem wielu paradygmatów programowania: obiektowego (klasy, dziedziczenie), imperatywnego oraz w pewnym stopniu funkcyjnego (praca na listach). Implementacje języka Python są dostępne dla wielu systemów operacyjnych oraz platform sprzętowych, w tym dla systemów wbudowanych.

```
def fib(n):
  a, b = 0, 1
  for _ in xrange(n):
    a, b = b, a+b
  return a
```

Rysunek 5.1: Kod funkcji w języku *Python* zwracającej n-ty wyraz ciągu Fibonacciego.

Wybrałem ten język programowania, ponieważ znałem go bardzo dobrze i potrafiłem się nim płynnie posługiwać. Drugim, również bardzo istotnym, powodem jest obecność wielu gotowych i przetestowanych bibliotek związanych z algorytmami uczenia maszynowego. Możliwość użycia pewnego, działającego i niezawodnego kodu przesądziła o wyborze tego języka.

Wysoka zwięzłość języka pozwoliła na eksperymentowanie z systemem komputerowym niewielkim kosztem czasu. Wprowadzanie małych zmian i poprawek mogło być realizowane ad-hoc, co znacznie przyspieszyło proces implementacji.

5.1.2 System kolejkowy RabbitMQ

Systemem odpowiedzialnym za przesyłanie komunikatów pomiędzy nadzorcą a robotnikami jest system kolejkowy *RabbitMQ*. Jest to otwarte oprogramowanie umożliwiające bezpieczne umieszczanie wiadomości w kolejce (na serwerze) w celu późniejszego ich odebrania przez zainteresowane procesy. Serwer *RabbitMQ* jest zaimplementowany w języku Erlang i w

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

pełni wspiera otwarty standard komunikacji *AMQP* (ang. *Advanced Message Queuing Protocol*). Twórcy tej technologii chwalą się wysoką wydajnością systemu, łatwością użycia oraz wsparciem ze strony innych firm.

Wybrałem właśnie ten system kolejkowy ze względu na jego przenośność (obsługuje systemy operacyjne z rodziny Windows, Linux i Mac OS) oraz bogate wsparcie dla programistów (obecność bibliotek w wielu językach programowania, w tym w Pythonie).

5.1.3 Biblioteka Celery

W celu uproszczenia kodu związanego z komunikacją z systemem kolejkowym, użyłem gotowej biblioteki o nazwie *Celery*. Biblioteka ta pozwala w prosty sposób zaimplementować elastyczny i niezawodny rozproszony system do obsługi dużej liczby wiadomości (zadań) jednocześnie oferując narzędzia do obsługi takiego systemu. *Celery* udostępnia nieskomplikowany interfejs programistyczny systemu kolejkowego, realizując faktyczną wymianę wiadomości za pomocą innych systemów tj. *RabbitMQ*, *Redis* lub *Mongo DB*.

```
from celery import Celery

app = Celery('tasks', broker='amqp://guest@localhost//')

@app.task
def add(x, y):
    return x + y
```

Rysunek 5.2: Kod przykładowego programu odbierającego zadania do policzenia sumy liczb (*Celery*). Kod związany z nasłuchiwaniem na nadchodzące zadania obliczeniowe przypomina prostą definicję funkcji w języku *Python*.

Projekt *Celery* jest przykładem otwartego oprogramowania, jest udostępniany na licencji BSD. Posiada duże zaplecze użytkowników oraz programistów. *Celery* jest biblioteką języka Python.

Użycie *Celery* pomogło utrzymać prostotę i przejrzystość kodu oraz umożliwiło bezproblemową komunikację z systemem *RabbitMQ*.

5.1.4 Baza danych PostgreSQL

Baza *PostgreSQL* jest systemem zarządzania relacyjnymi bazami danych z rozszerzeniami obiektowymi. Jest jednym z trzech najpopularniejszych rozwiązań wolnodostępnych z dziedziny baz danych. System ten jest rozwijany od połowy 1995 roku, całość jest zaimplementowana w języku *C*.

Baza ta została wybrana ze względu na wolny dostęp oraz dużą wydajność.

5.1.5 Mapowanie obiektowo-relacyjne SQLAlchemy

W celu uproszczenia odczytu i zapisu wartości do bazy danych, użyta została biblioteka *SQLAlchemy* udostępniająca warstwę mapowania obiektowo-relacyjnego. Głównym zadaniem mapowania obiektowo relacyjnego jest zapewnienie kontroli nad danymi w bazie za pomocą obiektów występujących w danym języku programowania. *SQLAlchemy* jest biblioteką dla języka *Python*.

Biblioteka *SQLAlchemy* pozwala w prosty sposób dodawać, wyszukiwać i usuwać wiersze z bazy danych. System ten jawnie oddziela operacje przeprowadzane na bazie od samej bazy danych. Możliwe jest zatem użycie tego samego kodu do współpracy z bazą *PostgreSQL*, *MySQL*, *Oracle*, *SQLite* lub nawet *Microsoft SQL Server*.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Biblioteka SQLAlchemy została wybrana aby uprościć implementację systemu komputerowego poprzez zdjęcie z programisty odpowiedzialności za prawidłowe konstruowanie kwarend. Z tego punktu widzenia biblioteka spełniła wszystkie oczekiwania.

5.1.6 Biblioteka scikit-learn

Kluczową częścią całego systemu jest implementacja algorytmów przetwarzania wstępnego i klasyfikacji. O ile niektóre fazy przetwarzania są nieskomplikowane i mogą być zaimplementowane własnoręcznie (imputacja, usuwanie atrybutów, skalowanie atrybutów) to przygotowanie kodu odpowiedzialnego za resztę algorytmów (grupowania i klasyfikacji) mogłoby zająć bardzo dużo czasu. Zważywszy, że celem pracy jest porównanie metod, a nie ich implementacja, w systemie została użyta gotowa biblioteka *scikit-learn* zawierająca implementację „najtrudniejszych" algorytmów.

Biblioteka *scikit-learn* jest biblioteką uczenia maszynowego dla języka *Python* rozpowszechnianą na licencji otwartego oprogramowania (licencja BSD). Biblioteka zawiera wiele implementacji popularnych algorytmów klasyfikacji, regresji i grupowania, w tym drzewa decyzyjne, lasy losowe, maszyny wektorów nośnych oraz wiele innych.

Użycie biblioteki *scikit-learn* pozwoliło na błyskawiczne użycie wielu algorytmów opisanych w pracy magisterskiej. Dodatkowo, dokumentacja biblioteki stanowi cenne źródło wiedzy na temat uczenia maszynowego.

5.2 Program liczący

Podrozdział ten koncentruje się na interesujących fragmentach implementacji programu-robotnika, liczącego oceny klasyfikacji dla rodzin algorytmów.

5.2.1 Buforowanie wyników

Analizując złożoność czasową każdego etapu algorytmu, można zauważyć, że najwięcej czasu poświęca się na obliczenia związane z grupowaniem oraz klasyfikowaniem danych. O ile klasyfikacja musi być przeprowadzana za każdym razem, o tyle grupowanie niekoniecznie. Dane po grupowaniu (czyli po przetworzeniu wstępnym) można zapamiętać oraz natychmiastowo odtworzyć w następnej iteracji przebiegu badań. Zapamiętywanie informacji o wynikach przetwarzania wstępnego może być zrealizowane poprzez przechowywanie danych w buforze. Bufor taki miałby ograniczoną pojemność, dlatego co jakiś czas należałoby usunąć z niego niepotrzebne wyniki obliczeń. Wyboru tych „niepotrzebnych" można dokonać algorytmem *LRU* (ang. *Least Recently Used*).

Klasa Cache

Buforowanie wyników zostało zrealizowane poprzez implementację dwóch klas: *LRUCache* oraz *Cache*. Pierwsza z klas to uniwersalna implementacja bufora opróżnianego zgodnie z algorytmem *LRU*. Druga klasa to bufor dedykowany pod kod związany z ocenianiem algorytmów przetwarzania i klasyfikacji. Klasa ta wewnętrznie używa klasy *LRUCache*. Implementację klasy *Cache* prezentuje listing 5.3.

Konstruktor klasy *Cache* przyjmuje parametr n oznaczający maksymalny rozmiar bufora. Klasę *Cache* można interpretować jako tablicę asocjacyjną z automatycznym usuwaniem wartości w której kluczem jest trójka wartości $(checksum, split, pd)$, gdzie $checksum$ to suma kontrolna pliku z danymi, $split$ to podział przykładów na zbiór trenujący i testowy, a pd to deskryptor przetwarzania wstępnego.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

```
class Cache(object):
    def __init__(self, n):
        self._cache = LRUCache(n)

    def contains(self, checksum, split, pd):
        return not self.get(checksum, split, pd) is None

    def set(self, checksum, split, pd, value):
        key = (str(checksum), str(split), str(pd))
        self._cache.add(key, value)

    def get(self, checksum, split, pd):
        key = (str(checksum), str(split), str(pd))
        return self._cache.get(key)
```

Rysunek 5.3: Implementacja klasy *Cache* w języku *Python*.

Metoda *contains* pozwala stwierdzić czy bufor zawiera dany klucz. Metoda *set* służy do ustalenia wartości pod określonym kluczem, a metoda *get* do pobrania wartości spod klucza.

Klasa LRUCache

Podstawą optymalizacji opisywanej w tej części rozdziału jest implementacja klasy odpowiedzialnej za algorytm *LRU*.

```
class LRUCache(object):
    class Node(object):
        def __init__(self, key, value):
            self.key = key
            self.value = value

    def __init__(self, n):
        self._n = n
        self._queue = []
        self._hash = {}

    def get(self, key):
        node = self._hash.get(key, None)
        if node:
            self._queue.remove(node)
            self._queue.insert(0, node)
            return node.value
        return None

    def add(self, key, value):
        if not key in self._hash:
            node = LRUCache.Node(key, value)
            self._queue.insert(0, node)
            self._hash[key] = node

            if len(self._queue) > self._n:
                removed_node = self._queue.pop()
                self._hash.pop(removed_node.key, None)
```

Rysunek 5.4: Implementacja klasy *LRUCache* w języku *Python*.

Klasa *LRUCache* przechowuje wewnętrzną kolekcję obiektów typu klucz-wartość. Konstruktor klasy przyjmuje pojedynczy argument n - rozmiar kontenera. Metoda *get* zwraca żądany element, jeżeli istnieje. Jeżeli element nie istnieje, zwracana jest wartość *None*. Metoda add dodaje do bufora nowy element. Jeżeli po dodaniu rozmiar bufora został przekroczony, obiekt klasy *LRUCache* automatycznie usuwa odpowiedni element.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

5.2.2 Wykonywanie obliczeń

Zastosowanie biblioteki *Celery* umożliwiło bardzo zwięzłą i przejrzystą implementację programu-robotnika. Dzięki szczegółowemu projektowi rozwiązania, który obejmuje warstwę zadań (deskryptory) oraz modele przetwarzania i klasyfikacji, wynikowy kod jest czytelny i przypomina istotę algorytmu, który reprezentuje (listing 5.5).

```
@app.task
def evaluate(filepath, random_state, pairs):
    checksum = file_checksum(filepath)
    xls = file_cache.get(filepath)

    scores = []
    splits = get_splits(xls, random_state)
    for pair in pairs:
        pd = pair.preprocessing_descriptor
        for split in splits:
            if not sample_cache.contains(checksum, split, pd):
                training, test = split

                # Training
                training_sample = Sample.from_file(xls, training)
                imputer = pd.impute(training_sample)
                training_sample.impute_nan(imputer)
                training_sample.remove_columns(pd.removed_columns)
                normalizer = training_sample.get_normalizer()
                training_sample.normalize(normalizer, pd.normalized_columns)
                clusterer = pd.cluster(training_sample)
                training_sample.merge(clusterer)

                # Test
                test_sample = Sample.from_file(xls, test)
                test_sample.impute_nan(imputer)
                test_sample.remove_columns(pd.removed_columns)
                test_sample.normalize(normalizer, pd.normalized_columns)
                test_sample.merge(clusterer)

                sample_cache.set(checksum, split, pd, (training_sample, test_sample))
            else:
                training_sample, test_sample = sample_cache.get(checksum, split, pd)

            clf = pair.classification_descriptor.create_classifier(training_sample)
            scores.append(get_scores(clf, training_sample, test_sample))
    return scores
```

Rysunek 5.5: Ciało funkcji *evaluate* odpowiedzialnej za liczenie ocen klasyfikacji dla bloków algorytmów.

Rozpoczęcie obliczeń związanych z oceną jakości klasyfikacji bloku algorytmów można traktować jako zdalne wywołanie procedury *evaluate* (*RPC*, ang. *Remote Procedure Call*). Parametrami takiego wywołania są ścieżka do pliku z danymi, obiekt przechowujący ziarno pseudolosowe oraz zbiór par deskryptorów (przetwarzania wstępnego i klasyfikacji).

W pierwszej kolejności, opierając się na liczbie przykładów w pliku oraz na ziarnie pseudolosowym, generowane jest 10 podziałów zbioru danych na zbiory trenujące i testowe (funkcja *get_splits()*).

Następnie dla każdej pary i dla każdego podziału zbioru tworzony jest obiekt t klasy *Sample* zawierający tylko te przykłady, które wchodzą w skład zbioru trenującego. Na podstawie tego obiektu tworzony jest model imputacji. Modelu używa się aby usunąć braki z obiektu t. Dalej z obiektu usuwane są wybrane atrybuty. Kolejnym krokiem jest utworzenie modelu skalowania używając obiektu t. W kolejnym kroku model ten służy do przeskalowania atrybutów obiektu t. Kiedy to nastąpi, wytwarzany jest model grupowania. Używa się go następnie do zgrupowania wybranych atrybutów w t.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Po wygenerowaniu wszystkich potrzebnych modeli na zbiorze trenującym i po przetworzeniu tego zbioru, przychodzi kolej na transformację zbioru testowego. Poddaje się go kolejno etapom imputacji, usuwania atrybutów, skalowania i grupowania.

Klasyfikacja danych odbywa się po przetworzeniu wstępnym zbioru uczącego i testowego. Najpierw, przy użyciu metody *create_classifier()*, tworzony jest wybrany klasyfikator clf. Nauka, test i ocena klasyfikatora clf odbywa się przy użyciu funkcji pomocniczej *get_scores()*. Wynik funkcji - trójka ocen predykcji - zostaje dodana do zbioru ocen.

Optymalizacja opisana w punkcie 5.2.1 polega na użyciu obiektu klasy *Cache*. Przed rozpoczęciem każdego przetwarzania wstępnego sprawdza się, czy wyniki znajdują się już w buforze. Jeżeli tak, stają się natychmiast dostępne i program przechodzi do oceny klasyfikacji. Jeżeli nie, program wykonuje przetwarzanie i utrwala (chwilowo) wyniki w buforze.

Funkcja *evaluate()* zwraca zbiór ocen, co w przypadku biblioteki *Celery* oznacza wysłanie tego zbioru do programu-nadzorcy.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

6. Badania

Rozdział ten poświęcony jest opisowi przebiegu badań oraz obserwacjom poczynionym na wynikach tych badań.

6.1 Przebieg badań

Badania przeprowadzono dwuetapowo. W pierwszym etapie pominięto fazę grupowania atrybutów ale uwzględniono fazę imputacji, usuwania i skalowania. Pominięcie grupowania było związane z dużym narzutem czasowym spowodowanym bardzo dużą liczbą dodatkowych kombinacji, które grupowanie wnosi do procesu obliczeń. Po analizie wyników z pierwszej fazy wyłoniono najkorzystniejsze warianty rodzin algorytmów i przystąpiono do etapu drugiego. W części tej użyto algorytmów grupowania dla tych rodzajów rodzin, które najlepiej wypadły w fazie pierwszej.

6.1.1 Faza pierwsza

Opis przebiegu badań

Badania w fazie pierwszej zostały przeprowadzone w następujący sposób. Dla każdego rozważanego algorytmu klasyfikacji (naiwny klasyfikator bayesowski, drzewo decyzyjne, las losowy, maszyna wektorów nośnych) oraz każdej metody imputacji (wartością średniej, wartością mediany) obliczone zostały wyniki następujących rodzin algorytmów:

1. Rodzina bez pozostałych etapów przetwarzania wstępnego
2. Rodzina z usuwaniem wszystkich kombinacji atrybutów
3. Rodzina ze skalowaniem wszystkich kombinacji atrybutów
4. Rodzina z usuwaniem i skalowaniem wszystkich kombinacji atrybutów

Dla tak policzonych zbiorów ocen jakości klasyfikacji obliczono trzy rankingi:

1. Wg wrażliwości
2. Wg precyzji
3. Wg metryki F1

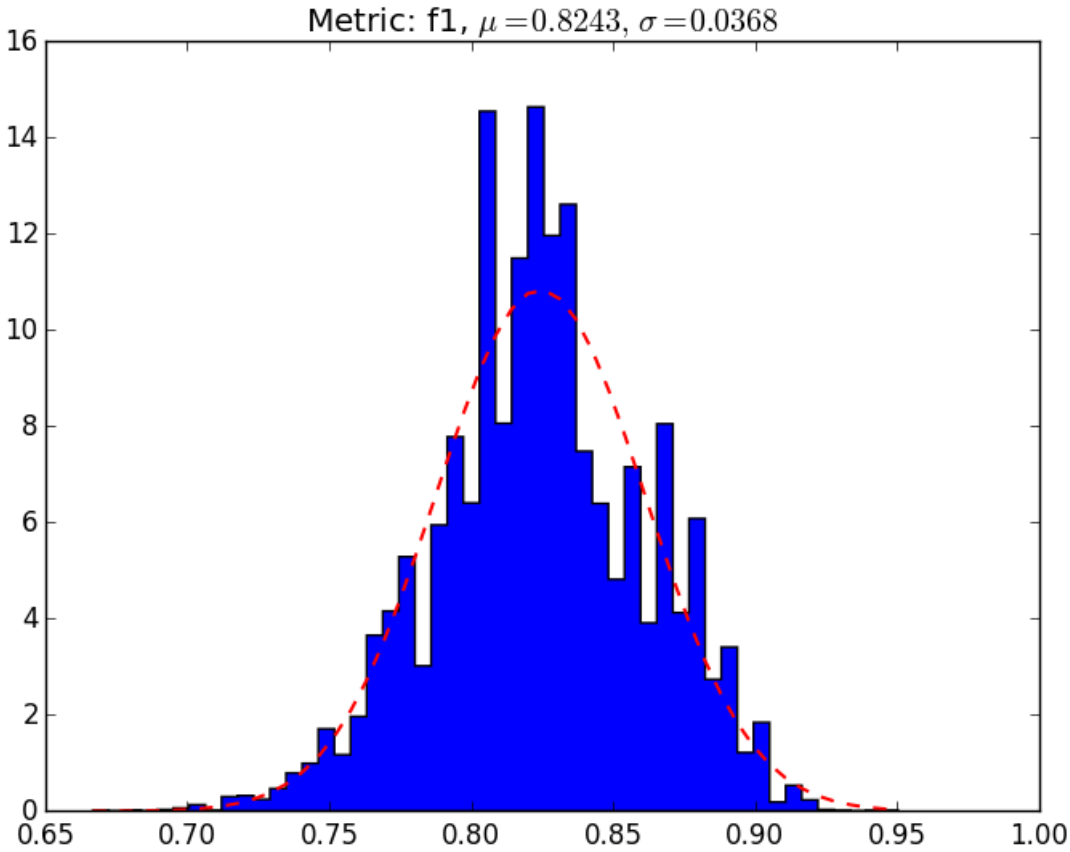
Rezultaty badań (kolejność rodzin w rankingach, wzajemne relacje rodzin w rankingach, wartości odpowiednich miar jakości klasyfikacji) poddano analizie.

Analiza najlepszych rodzin w rankingach

Dla danych opisanych w punkcie 3.1.1 w rankingu miar F1 najlepszą rodziną okazała się ta, która używała lasu losowego, skalowała wszystkie kombinacje atrybutów oraz imputowała brakujące wartości średnią. Rodzina ta zajęła jednocześnie drugie miejsce w rankingu precyzji i wrażliwości. Średnia wartość miary F1 wynosiła 0.8243, a odchylenie standardowe było równe 0.0368. Rysunek 6.1 prezentuje histogram miar F1 dla tej rodziny.

Do ciekawych obserwacji można było dojść przyglądając się najlepszym dziesięciu rodzinom algorytmów. Aż sześć z nich używało klasyfikatora las losowy, a pozostałe cztery drzewa

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00



Rysunek 6.1: Histogram miar oceny F1 dla najlepszej rodziny algorytmów w rankingu F1. Linia przerywana to kontur zbliżonego rozkładu normalnego.

decyzyjnego. Poza tym, połowa algorytmów imputowała braki medianą, a druga połowa wartością średnią.

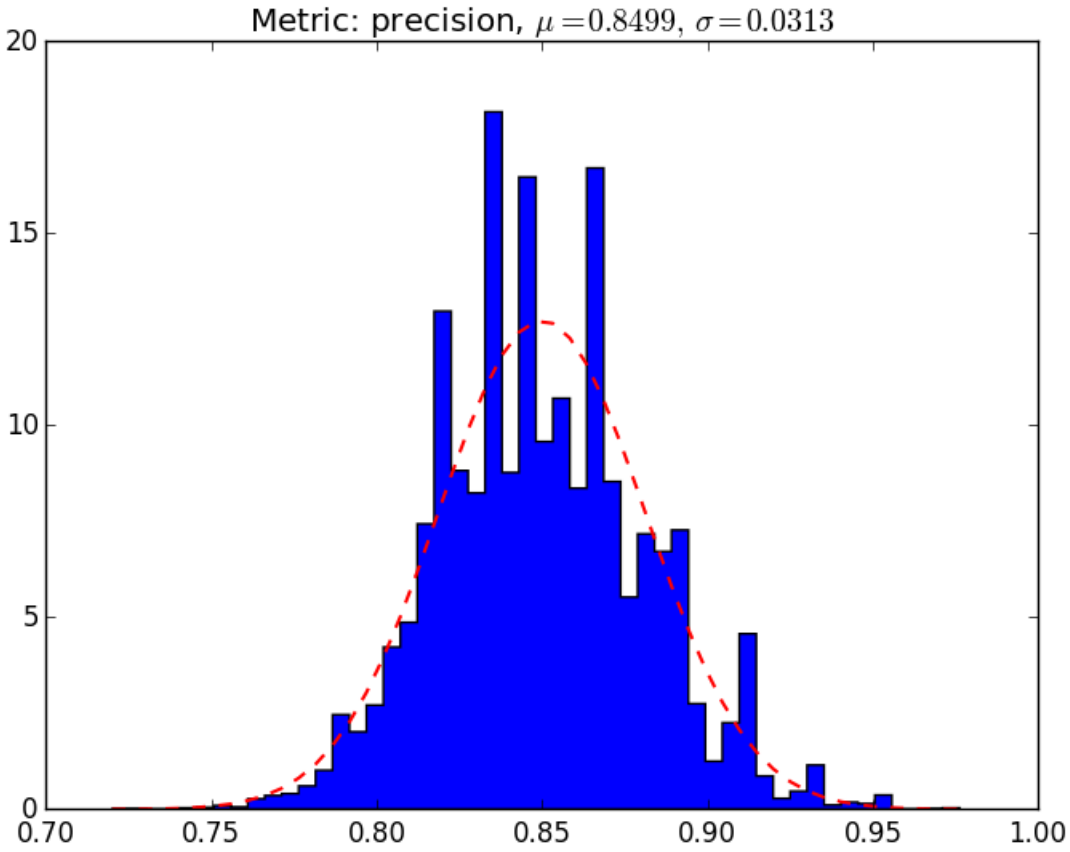
W rankingu oceny precyzji klasyfikacji za najlepszą rodzinę została uznana ta, która używała lasu losowego, skalowała kombinacje atrybutów i imputowała braki wartością mediany (średnia ocen 0.8499, odchylenie 0.0313). Co ciekawe, rodzina ta zajęła drugie miejsce w rankingu miary F1. Histogram precyzji rodziny pokazano na rysunku 6.2.

Podobnie jak w przypadku rankingu miary F1, pierwsze 10 miejsc rankingu zostało zajętych przez rodziny używające klasyfikatora las losowy lub drzewo decyzyjne. Siedem na dziesięć rodzin użyło lasu losowego, trzy na dziesięć drzewa. Dodatkowo, tak jak poprzednio, połowa dziesięciu najlepszych rodzin imputowała medianą, a druga połowa średnią.

Zwycięzcą rankingu wrażliwości została rodzina używająca maszyny wektorów nośnych jako klasyfikatora i imputująca wartością średniej. Średnia wartość wrażliwości w rodzinie wynosiła 0.9605, a odchylenie standardowe było równe 0.0773. Histogram wrażliwości rodziny prezentuje ilustracja 6.3.

Cztery spośród dziesięciu najlepszych rodzin używało lasu losowego, trzy drzewa decyzyjnego, a ostatnie trzy maszyny wektorów nośnych. Cztery na dziesięć rodzin imputowało medianą, sześć wartością średniej.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00



Rysunek 6.2: Histogram precyzji dla najlepszej rodziny algorytmów w rankingu precyzji. Linia przerywana to kontur zbliżonego rozkładu normalnego.

Analiza kolejności rodzin w rankingach

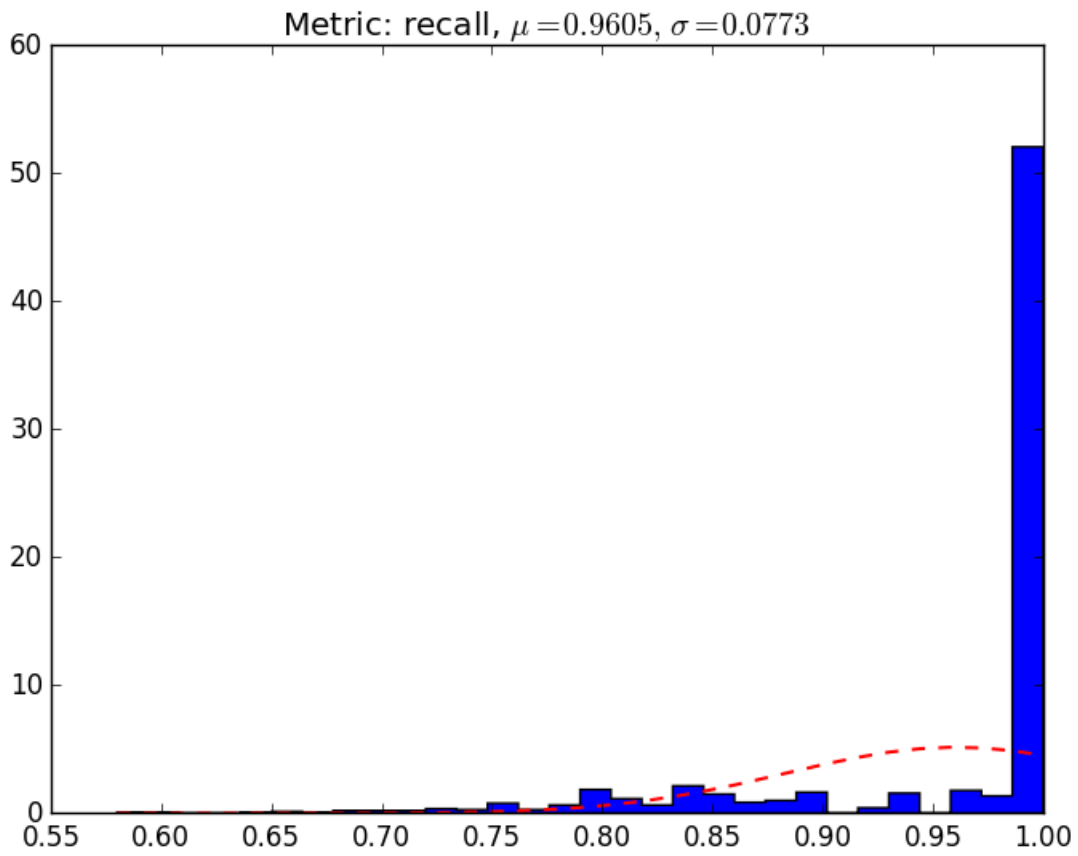
Następna część analizy rankingów skupiła się na wpływie metod przetwarzania wstępnego na końcową pozycję rodziny w rankingu. Dla danej metody klasyfikacji badane były pozycje rodzin używające różnych kombinacji etapów wstępnego przetwarzania. Pozycje rodzin w rankingu przedstawia tabela 6.1.

Dla wszystkich klasyfikatorów w rankingu F1 skalowanie atrybutów działa lepiej niż skalowanie połączone z usuwaniem atrybutów. To zaś działa zawsze lepiej niż usuwanie kombinacji atrybutów. W rankingu precyzji rodziny usuwające i skalujące atrybuty górują nad rodzinami, które tylko usuwają atrybuty. Spoglądając na wyniki wg rankingu wrażliwości, skalowanie atrybutów zawsze działa lepiej niż usuwanie lub usuwanie i skalowanie atrybutów.

Niezależnie od tego, jaki rodzaj oceny chcemy maksymalizować, dla lasu losowego skalowanie zawsze sprawuje się lepiej niż skalowanie z usuwaniem bądź samo usuwanie atrybutów. Podobnie w przypadku klasyfikatora, jakim jest drzew decyzyjne. W tym przypadku widać jednak jeszcze jedną regułę: skalowanie atrybutów działa lepiej niż prosty klasyfikator bez skalowania. Naiwny klasyfikator bayesowski, podobnie jak poprzednie, działa najlepiej gdy użyje się go ze skalowaniem atrybutów. Nie można niestety nic powiedzieć na temat generalnego wpływu etapów przetwarzania na jakość działania maszyny wektorów nośnych.

Powyższe wyniki jasno pokazują, że skalowanie atrybutów ma korzystny wpływ na jakość klasyfikacji dostępnych danych.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00



Rysunek 6.3: Histogram wrażliwości dla najlepszej rodziny algorytmów w rankingu wrażliwości. Linia przerywana to kontur zbliżonego rozkładu normalnego.

Analiza całościowa

Rozkłady ocen poszczególnych rodzin algorytmów były same w sobie źródłem cennych informacji. Okazało się, że rodziny algorytmów używające naiwnego klasyfikatora bayesowskiego i maszyny wektorów nośnych mają bardzo duże dysproporcje pomiędzy ocenami wrażliwości i precyzji. Maszyna wektorów nośnych dawała bardzo wysokie wyniki wrażliwości (ok. 91%), ale bardzo niskie wartości precyzji (ok. 56%). Klasyfikator bayesowski odznaczał się natomiast bardzo niską wrażliwością (ok. 13%). Tak niskie wartości ocen dyskwalifikują te dwa klasyfikatory. Przykładowe statystyki dla wybranych rodzin prezentuje tabela 6.2.

6.1.2 Faza druga

Opis przebiegu badań

Analiza wyników z fazy pierwszej pokazała, że skalowanie atrybutów jest uniwersalną metodą polepszenia jakości klasyfikacji. W związku z tym, faza druga polegała na przeprowadzeniu etapów grupowania atrybutów dla uprzednio przeskalowanych atrybutów.

Dla każdej metody klasyfikacji (naiwny klasyfikator bayesowski, drzewo decyzyjne, las losowy, maszyna wektorów nośnych) oraz każdej metody imputacji obliczone zostały wyniki następujących rodzin:

1. Jednokrotne grupowanie algorytmem *K-Średnich* kombinacji przeskalowanych atrybutów o maksymalnej liczności równej 2.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Ranking	Klasyfikator	Kolejność rodzin
F1	Las losowy	pure, scale > remove+scale, remove
	Drzewo decyzyjne	scale > pure, remove+scale > remove
	Bayes	scale > remove+scale > remove > pure
	SVM	pure > scale > remove+scale > remove
Precyzja	Las losowy	scale > pure > remove+scale > remove
	Drzewo decyzyjne	scale > remove+scale > pure > remove
	Bayes	scale > pure > remove+scale > remove
	SVM	remove+scale > scale, remove > pure
Wrażliwość	Las losowy	pure, scale > remove+scale > remove
	Drzewo decyzyjne	scale > pure, remove+scale > remove
	Bayes	scale > remove+scale > remove > pure
	SVM	pure > scale > remove > remove+scale

Tablica 6.1: Kolejność w rankingach rodzin używających różnych kombinacji etapów wstępnego przetwarzania. Oznaczenia: *pure* - imputacja, *remove* - imputacja z usuwaniem atrybutów, *scale* - imputacja ze skalowaniem atrybutów, *remove+scale* - imputacja z usuwaniem i skalowaniem atrybutów.

Klasyfikator	Wrażliwość	Precyzja
Bayes	0.1355 ± 0.0427	0.7238 ± 0.1130
Bayes	0.1413 ± 0.0402	0.7626 ± 0.1451
Bayes	0.1318 ± 0.1050	0.7101 ± 0.1743
SVM	0.9205 ± 0.0773	0.5551 ± 0.0899
SVM	0.8902 ± 0.1693	0.5692 ± 0.0918
SVM	0.8904 ± 0.1688	0.5698 ± 0.0922

Tablica 6.2: Wartości średniej ocen i odchylenia standardowego ocen dla wybranych rodzin algorytmów używających naiwnego klasyfikatora bayesowskiego (*Bayes*) albo maszyny wektorów nośnych (*SVM*).

2. Jednokrotne grupowanie algorytmem *Mean Shift* kombinacji przeskalowanych atrybutów o maksymalnej liczności równej 2.

Rezultaty badań poddano analizie.

Obserwacje

Po potrzymaniu wyników okazało się, że działanie każdego algorytmu klasyfikacji wg każdego rankingu pogorszyło się po dodaniu etapu grupowania.

6.2 Wnioski z badań

Analiza wyników i obserwacji prowadzi do następujących wniosków.

Mając na celu maksymalizację precyzji lub miary F1, najlepszym wyborem jest rodzina algorytmów używająca lasu losowego jako klasyfikatora oraz skalująca wartości wszystkich atrybutów. Metody te pozwalają osiągnąć wynik miary F1 rzędu 82% oraz precyzji rzędu 85%. Co ciekawe, wybór metody imputacji nie wpływa znacząco na ostateczną jakość predykcji.

Drugiem wnioskiem jest to, że rodziny używające lasu losowego lub drzewa decyzyjnego uzyskują najlepsze wyniki wszystkich rozpatrywanych miar jakości oceny (precyzji, wrażliwości, F1).

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Kolejnym wnioskiem jest pozytywny wpływ skalowania atrybutów na działanie drzewa decyzyjnego i naiwnego klasyfikatora bayesowskiego, bez względu na rozpatrywaną metrykę jakości oceny klasyfikacji.

W odróżnieniu od wyżej wymienionego skalowania, usuwanie atrybutów bardzo często prowadzi do pogorszenia jakości klasyfikacji. Co więcej, grupowanie atrybutów zawsze pogarsza ocenę.

Ostatecznie, naiwny klasyfikator bayesowski oraz maszyna wektorów nośnych, ze względu na niskie wartości wrażliwości i predykcji, nie nadają się do klasyfikowania posiadanych danych medycznych.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

7. Podsumowanie

Po zaimplementowaniu systemu komputerowego i przeprowadzeniu na nim badań, stwierdzam, że cele pracy magisterskiej zostały w całości zrealizowane. Projekt systemu, obejmujący architekturę rozwiązania, budowę biblioteki obliczeniowej *mltool* oraz schematy działania pozostałych komponentów, wyraźnie uprościł późniejszą implementację. Użycie deskryptorów algorytmów umożliwiło proste prowadzenie obliczeń w sposób równoległy.

Przygotowany system komputerowy spełnia założenia pracy. Możliwe jest badanie jakości klasyfikacji dla definiowanych przez użytkownika rodzin algorytmów. System pozwala porównywać rodziny ze sobą, wizualizować rozkład ocen w rodzinach oraz porządkować rodziny wg wybranej miary oceny predykcji. Aplikacja posiada interfejs konsolowy pozwalający na przeprowadzanie wszystkich niezbędnych operacji.

Zrealizowany system obliczeniowy może być w przyszłości ulepszany i rozbudowywany. Modularna budowa systemu nakierowana na obliczenia rozproszone pozwala na łatwe dostosowanie systemu do pracy w klastrze komputerowym lub w chmurze obliczeniowej. Użyte technologie (język programowania *Python*, system kolejkowy *RabbitMQ* oraz biblioteka *Celery*, baza danych *PostgreSQL*) są popularne i aktywnie rozwijane, dzięki czemu zainteresowane osoby mogą z łatwością wprowadzać własne zmiany do systemu.

Badania przeprowadzone w trakcie pisania pracy doprowadziły do wielu wniosków. Po pierwsze, skalowanie danych i ich późniejsza klasyfikacja metodą lasu losowego danej bardzo dobre wyniki w kontekście precyzji oraz miary F1. Po drugie, las losowy oraz drzewa decyzyjne sprawdzają się dobrze dla każdego rodzaju oceny. Kolejną obserwacją jest pozytywny wpływ skalowania atrybutów na działanie drzewa decyzyjnego i naiwnego klasyfikatora bayesowskiego, bez względu na rodzaj oceny predykcji. Badania pozwoliły również odkryć, że usuwanie atrybutów oraz ich grupowanie zazwyczaj prowadzi do pogorszenia oceny. Analizując wyniki obliczeń udało się ustalić, że naiwny klasyfikator bayesowski oraz maszyna wektorów nośnych nie nadają się do pracy z posiadanym zbiorem danych.

Poprawność działania systemu jest do pewnego stopnia zagwarantowana obecnością wielu testów automatycznych. W celu całkowitego upewnienia się co do słuszności wyników, należy przeprowadzić dodatkowe badania dla znanych danych. Wyniki eksperymentów należy porównać z obserwacjami opisanymi w literaturze. Badania tego typu nie zostały przeprowadzone ale są planowane w przyszłości.

Praca magisterska może okazać się przydatna dla osób, które mają zamiar rozpocząć swoją przygodę z bardzo ciekawą dziedziną informatyki, jaką jest uczenie maszynowe, a w szczególności klasyfikacja i przetwarzanie wstępne.

Prezentacja niniejszej pracy była częścią trzydziestego szóstego sympozjum IEEE w Wildze w 2015 roku.

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00

Bibliografia

[1] Karolina Dębska, *Występowanie raka piersi w Polsce*, portal abczdrowie.pl, Maj 2015, https://portal.abczdrowie.pl/wystepowanie-raka-piersi-w-polsce

[2] *Nowotwory piersi u kobiet (C50)*, Krajowy rejestr nowotworów, Maj 2015, http://onkologia.org.pl/nowotwory-piersi-kobiet/

[3] *Klasyfikacja statystyczna*, Wikipedia, Maj 2015, http://pl.wikipedia.org/wiki/Klasyfikacja_statystyczna

[4] *Preprocessing data*, podręcznik użytkownika biblioteki *scikit-learn*, Maj 2015, http://scikit-learn.org/stable/modules/preprocessing.html#preprocessing

[5] *Principal component analysis*, Wikipedia, Maj 2015, http://en.wikipedia.org/wiki/Principal_component_analysis

[6] Paweł Cichosz, *Systemy Uczące Się*, Wydawnictwa Naukowo-Techniczne, Warszawa, 2000

[7] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, *Design Patterns, Elements of Reusable Object-Oriented Software*

Pobrano z https://repo.pw.edu.pl / Downloaded from Repository of Warsaw University of Technology 2025-11-06 11:00